FLARE: VERIFYING MILP REFORMULATIONS WITH LLM-BASED THEOREM PROVING

Henry Robbins
Stanford University
hwr@stanford.edu

Connor Lawless
Stanford University
lawlessc@stanford.edu

Madeleine Udell
Stanford University
udell@stanford.edu

Ellen Vitercik
Stanford University
vitercik@stanford.edu

ABSTRACT

Mixed-Integer Linear Programming (MILP) is a fundamental tool for combinatorial optimization with extensive real-world applications. A central challenge is designing efficient MILP formulations. Large Language Models (LLMs) offer new opportunities to automate the modeling process, from deriving formulations to strengthening them. To ensure correctness, we need robust methods to compare formulations. However, existing approaches evaluate formulations numerically and fail to reason about general problem instances. We resolve this limitation by introducing a constructive notion of MILP reformulation that can be formalized in Lean and machine-checked. We develop FLARE[1] (Formulation-Level Automated Reformulation Evaluation), a method that uses an LLM-based agent and the Lean proof assistant to verify proposed reformulations against a reference. To evaluate our approach, we introduce FormulationBench[2], a challenging dataset of 20 problems and 109 formulations. FLARE outperforms existing methods, with **100%** accuracy on the NP-hard subset of FormulationBench. Furthermore, FLARE produces a machine-checkable certificate for every reformulation it accepts. For cases where formal guarantees are not necessary, we introduce FLARE-NL, a fast and cheap LLM proxy that matches FLARE's accuracy but produces no certificate.[3] These methods enable reliable verification in automated optimization modeling.

Keywords Mixed Integer Linear Programming · Automated Theorem Proving · Large Language Models

1 Introduction

Mixed-Integer Linear Programming (MILP) is a fundamental tool for combinatorial optimization with applications in scheduling [17, 32], planning [48], energy [45, 31], and chip design [53]. A central challenge in MILP is problem formulation: translating a real-world scenario into a concrete mathematical model. Historically, problem formulation has required significant technical expertise. However, recent work has highlighted the ability of LLMs to translate natural-language problem descriptions into MILP formulations [63, 1, 3, 25, 9]. The primary objective is to generate MILP formulations that faithfully represent the underlying optimization problem.

Developing a faithful formulation is just the first stage in the modeling process. The same problem can often be expressed by multiple formulations which vary dramatically in solve times. In practice, experts use techniques like reformulation [56], decomposition [61], and cutting planes [43] to obtain efficient MILP formulations. LLMs offer the potential to automate this process [66, 16]. More generally, we can view optimization modeling as a special case of algorithm design, where the chosen model dictates the runtime of the algorithm. In the broader algorithm design context, LLMs have recently been used to iteratively evolve efficient algorithms through generation and evaluation [52, 41, 67, 47]. This approach has led to advances in mathematical discovery [22], vehicle routing [24, 64], and system design [23].

Key Challenges. A key challenge in automated algorithm design, particularly acute in the context of optimization modeling, is to ensure that an AI-generated variant certifiably solves the original problem. Existing approaches rely heavily on heuristics, most commonly comparing optimal objective values on a single instance [1, 25, 37]. However,

[1] FLARE is implemented in the `milp-flare` Python package; see https://flare.henryrobbins.com.
[2] Download via the `formulation-bench` Python package; see https://formulation-bench.henryrobbins.com.
[3] All experimental code is available at https://github.com/henryrobbins/flare.

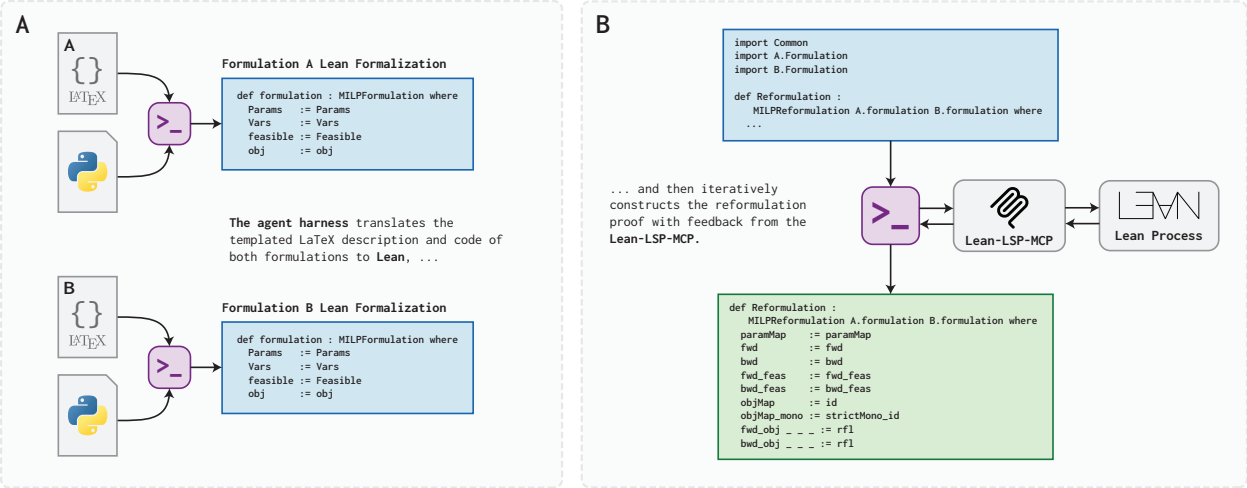

Figure 1: The FLARE workflow. (a) An agent is given templated LaTeX and Python representations of a pair of MILP formulations and their parameter mapping and is instructed to formalize them in Lean. Combined with our formalization of MILP reformulation, this yields a formal claim that formulation B is a reformulation of A under the fixed parameter map. (b) In the second phase, the agent attempts to construct a Lean proof of that claim. The Lean-LSP-MCP allows the agent to obtain detailed feedback from the Lean process as it develops the proof.

such checks are unreliable and can fail under simple transformations such as the addition of cutting planes or rescaling an objective (see [68] for a discussion).

Inspired by Karp reductions in complexity theory, recent work by Zhai et al. [68] introduced EquivaMap, which uses LLMs to discover how solutions map between formulations. However, its guarantees remain *instance-level*. It validates a reformulation for a specific instance (e.g., set of jobs to schedule) but not in general (e.g., any possible set of jobs to schedule). Such instance-level checks can miss formulation inconsistencies that arise on other problem instances. For example, this paper identifies several cutting planes proposed by an LLM-based modeling framework called EvoCut [66] that pass instance-level validation but remove optimal solutions for some instances. To mitigate this risk, we seek *formulation-level* guarantees satisfied by all problem instances. Such guarantees require reasoning over all instances, rather than computationally evaluating a small subset of instances.

Our Contributions. This paper introduces FLARE, an automated framework for validating reformulations at the formulation-level. FLARE uses formal verification to make universal claims over problem instances machine-checkable, exploiting recent advances in automated theorem proving (ATP) to generate reformulation proofs automatically [42].

Classic notions of reformulation require reasoning about optimal solution sets and are not well-suited to ATP. To address this challenge, we introduce and formalize a constructive definition of reformulation that requires explicit mappings between parameter spaces, feasible regions, and objective values. FLARE substitutes autoformalized MILP formulations into this definition to obtain a formal statement that can be certified with ATP. Conditioned on faithful formalization, a certificate proves a reformulation is valid for all problem instances. If ATP fails to produce a certificate, FLARE refuses to certify the reformulation as valid. FLARE is the first automated approach to produce verifiable reformulation certificates, enabling trustworthy optimization modeling with LLMs.

We also introduce FLARE-NL, a fast and cheap proxy that prompts a frontier reasoning model with the same definition. The two methods serve different regimes. A FLARE certificate on faithful formalizations admits no false positives; this property is critical in settings with zero tolerance for error (e.g., energy [31]). FLARE-NL is faster and cheaper but offers no such guarantee, making it the natural choice in non-critical settings or as a screening heuristic prior to FLARE.

Our contributions can be summarized as follows:

- **Constructive definition of MILP reformulation.** We introduce a constructive definition of MILP reformulation that is amenable to formal verification and ATP, enabling the first automated method for validating reformulations at the formulation level.
- **AI for formulation verification.** We develop FLARE, a framework that combines LLM-based autoformalization with ATP to generate machine-checkable reformulation certificates, and FLARE-NL, an LLM proxy that trades formal guarantees for improved speed and cost.

- **Realistic benchmark for reformulation.** We introduce FormulationBench, a benchmark dataset of 20 problems and 109 formulations capturing realistic modeling transformations. Empirically, we show both FLARE and FLARE-NL outperform existing methods, achieving **100%** accuracy on the NP-hard subset of FormulationBench.
- **Demonstrated need for formal proofs.** We verify prior AI-driven MILP reformulations, revealing 5 invalid cutting planes proposed by EvoCut [66] and 4 reformulations proposed in Ferchtandiker et al. [16]: these "equivalent" formulations are wrong or omit necessary assumptions (Appendix F).

2 Related Work

An efficient MILP formulation can decrease the solve time by orders of magnitude [11]. To improve solve time, experts apply transformations such as lifting [6], change of variables [61], and cutting planes [55] that strengthen the formulation while preserving optimal solutions (see [38] for a broader discussion). We study the complementary problem of *verifying* such reformulations, particularly in emerging settings where formulations are generated or modified by LLMs. This perspective connects three lines of research: (i) LLM-based systems that generate and refine MILP formulations, (ii) methods for verifying reformulations, and (iii) autoformalization and automated theorem proving.

LLMs for MILP Modeling. A growing body of work has applied LLMs to automate translating natural language into a MILP formulation. Early efforts focused on natural language processing (NLP) pipelines for structured extraction [49], while more recent *optimization copilots* use LLM-based multi-agent systems [1, 2, 46, 63, 37, 3, 15] or fine-tuning [25, 28] to handle complex, multi-constraint problems. These capabilities have been applied across domains including supply chain management [36], diagnosing infeasible models [8, 9], and scheduling [33]. Beyond formulating a *correct* model, recent work has also explored the use of LLMs to generate *stronger* MILP formulations via cutting planes [66], reformulations [16], or better solver configuration [34]. However, these approaches rely on instance-level validation. We demonstrate examples where this limitation leads to invalid cutting planes and reformulations.

Automatic Reformulation Checking Methods. Determining whether one formulation is a reformulation of another is central to evaluating LLM-generated MILP formulations. Existing approaches largely operate at the instance level. Canonical accuracy checks for a direct mapping between declarations (i.e., constraints and objectives) [49], while execution accuracy compares optimal objective values after solving both formulations [1, 25, 37]. More recently, *EquivaMap* [68] used LLMs to infer variable mappings between formulations and validate them by mapping optimal solutions on a specific instance. Structural approaches represent MILPs as bipartite graphs and measure similarity via graph isomorphism or edit distance [65, 54, 59, 60], avoiding the need to solve the optimization problem directly. Unlike existing approaches, FLARE verifies reformulations at the formulation-level by producing machine-checkable certificates that hold across all problem instances.

Autoformalization and Automated Theorem Proving. Autoformalization translates natural language into formal representations [62, 20, 58, 40, 27], while automated theorem proving (ATP) generates machine-checkable proofs for formal statements [39, 50, 57, 10, 5, 26, 42, 51]. Recent advances in LLMs have significantly improved both tasks, with agentic frameworks combining (fine-tuned) language models and proof assistants (e.g., Lean) to formalize and solve complex mathematical problems. Most recently, simple agentic harnesses have been shown to be competitive ATP methods [51, 42]. We adopt an LLM-based agent as the ATP method in FLARE and introduce a constructive definition of reformulation to make proving reformulations tractable.

3 Formalizing Formulations

We begin by formally defining MILP formulation. Here, we make a clear distinction between a *formulation* and the parameter (or data) values that define a particular *instance* of the problem [18]. Take the traveling salesman problem (TSP) as an example. A TSP formulation is defined on an abstract set of cities. A TSP instance is one such city set. Instantiating the formulation with an instance yields a concrete MILP to be solved.

Definition 3.1. A MILP *formulation* $\mathcal{M}$ is a tuple $\mathcal{M} = (\mathcal{P}, \mathcal{F}, f_0)$ with parameter space $\mathcal{P}$, feasible region $\mathcal{F}(p) \subseteq \mathbb{R}^{n(p)}$, and objective function f_0. For instance $p \in \mathcal{P}$, the feasible region $\mathcal{F}(p)$ is defined by $m(p)$ linear constraints, $f_i(\cdot; p) : \mathbb{R}^{n(p)} \to \mathbb{R}$ for all $i \in [m(p)]$. The first $k(p) \leq n(p)$ variables are integers. The feasible region is

$$\mathcal{F}(p) = \{x \in \mathbb{Z}^{k(p)} \times \mathbb{R}^{n(p)-k(p)} \mid f_i(x; p) \leq 0 \; \forall i \in [m(p)]\}.$$

When the instance p is clear from context, we use n, m, and k instead of the parameterized forms. The objective is to minimize[1] the linear function $f_0(\cdot; p) : \mathbb{R}^{n(p)} \to \mathbb{R}$. A formulation $\mathcal{M}$ is *instantiated* with an instance $p \in \mathcal{P}$. We denote an instantiated formulation as $\mathcal{M}(p) = (\mathcal{F}(p), f_0(p))$.

[1]We write all formulations as minimization problems; maximization objectives can be converted by negating the objective.

Running Example. The traveling salesman problem (TSP) aims to find the shortest tour in a graph that visits every node exactly once. A TSP instance is a weighted fully-connected graph $G = (V, E, w)$ on $n = |V| \geq 2$ nodes with edge weights w_{ij}. We write $\mathcal{P}_{\text{TSP}}$ for the set of all such graphs. Two common MILP formulations for the TSP are the Dantzig-Fulkerson-Johnson (DFJ) [12] and Miller-Tucker-Zemlin (MTZ) [44] formulations.

$$
\begin{aligned}
\min \quad & \sum_{(i,j)\in E} w_{ij}\, x_{ij} \\
\text{s.t.} \quad & \sum_{j \in V\setminus\{i\}} x_{ij} = 1 && \forall i \in V \\
& \sum_{i \in V\setminus\{j\}} x_{ij} = 1 && \forall j \in V \\
& \sum_{i\in S}\sum_{j\in S} x_{ij} \leq |S| - 1 && \forall S \subset V,\ 2 \leq |S| \leq n-1 \\
& x_{ij} \in \{0,1\} && \forall (i,j) \in E
\end{aligned}
\qquad \text{(DFJ)}
$$

$$
\begin{aligned}
\min \quad & \sum_{(i,j)\in E} w_{ij}\, x_{ij} \\
\text{s.t.} \quad & \sum_{j \in V\setminus\{i\}} x_{ij} = 1 && \forall i \in V \\
& \sum_{i \in V\setminus\{j\}} x_{ij} = 1 && \forall j \in V \\
& u_i - u_j + n\, x_{ij} \leq n - 1 && \forall i, j \in V \setminus \{1\},\ i \neq j \\
& u_1 = 1 \\
& 2 \leq u_i \leq n && \forall i \in V \setminus \{1\} \\
& x_{ij} \in \{0,1\} && \forall (i,j) \in E
\end{aligned}
\qquad \text{(MTZ)}
$$

Both formulations define decision variables $x_{ij} \in \{0, 1\}$ to indicate if the edge (i, j) is used in the tour. To prevent subtours, DFJ uses an exponential family of subtour-elimination constraints, while MTZ uses a polynomial family of constraints with auxiliary position variables $u_i \in \mathbb{R}$ for each node. We denote these formulations as $\mathcal{M}_{\text{DFJ}}$ and $\mathcal{M}_{\text{MTZ}}$, respectively. Both formulations faithfully represent the TSP, but have notable differences that affect solve times. $\mathcal{M}_{\text{DFJ}}$ is a *stronger* formulation than $\mathcal{M}_{\text{MTZ}}$ in that it admits fewer fractional solutions in the linear relaxation. Despite the exponential size of $\mathcal{M}_{\text{DFJ}}$, it can be solved efficiently with constraint generation.

With a formal definition of MILP formulation established, we now consider multiple formal notions of reformulation and introduce a constructive definition amenable to formalization and ATP.

4 Formalizing Reformulations

Until now, we have used the term *reformulation* informally. It has an intuitive operational meaning: $\mathcal{M}'$ is a reformulation of $\mathcal{M}$ if one can map an instance $\mathcal{M}(p)$ to an instance $\mathcal{M}'(p')$, solve $\mathcal{M}'(p')$, and efficiently recover an optimal solution to $\mathcal{M}(p)$. Importantly, this is a *formulation-level* claim that holds across *all* problem instances $p \in \mathcal{P}$.

In Section 4.1, we review an existing notion of reformulation capturing this intuition and discuss why this definition is difficult to verify computationally. This limitation has led to proxies for reformulation that are easier to computationally verify but lack formulation-level guarantees (Section 4.2). Our key idea is to utilize tools from formal verification to make formulation-level reformulation claims machine-checkable. To this end, we propose a constructive definition of reformulation that is amenable to formalization and tractable for ATP (Section 4.3).

4.1 Existing Definition

Audet et al. [4] capture our intuitive notion of reformulation with a complexity-theoretic definition inspired by polynomial time Turing reductions [21].

Definition 4.1 (Audet Reformulation [4])**.** Let $\mathcal{M}$ and $\mathcal{M}'$ be two formulations with parameter spaces $\mathcal{P}$ and $\mathcal{P}'$. $\mathcal{M}'$ is an *Audet reformulation* of $\mathcal{M}$ if there exists a mapping $\Phi_{\text{p}} : \mathcal{P} \to \mathcal{P}'$ such that, for any instance $p \in \mathcal{P}$: if $\mathcal{M}(p)$ has an optimal solution, then $\mathcal{M}'(\Phi_{\text{p}}(p))$ also has an optimal solution, and every optimal solution to $\mathcal{M}'(\Phi_{\text{p}}(p))$ can be mapped back to an optimal solution of $\mathcal{M}(p)$ in polynomial time.

Remark 4.2. This definition is only meaningful when computing the optimal solution to both formulations is NP-hard and both formulations admit a feasible point for at least one parameter. The polynomial-time restriction on the optimal solution mapping excludes trivial mappings that solve $\mathcal{M}(p)$ directly.

Remark 4.3. The Audet reformulation places no complexity restriction on the parameter mapping Φ_{p} itself.

TSP Example. Observe that $\mathcal{M}_{\text{MTZ}}$ is an Audet reformulation of $\mathcal{M}_{\text{DFJ}}$. The mapping Φ_{p} is the identity as both formulations use the same parameter space $\mathcal{P}_{\text{TSP}}$. Given an optimal solution of $\mathcal{M}_{\text{MTZ}}(\Phi_{\text{p}}(p))$, we can obtain an optimal solution to $\mathcal{M}_{\text{DFJ}}(p)$ by mapping x_{ij} to itself and dropping the position variables u_i, a linear time operation.

This definition captures a formulation-level notion of reformulation as desired. However, it is difficult to verify computationally since it quantifies over all instances $p \in \mathcal{P}$. Furthermore, ATP must identify how an optimal solution to $\mathcal{M}'(\Phi_{\text{p}}(p))$ can recover an optimal solution to $\mathcal{M}(p)$. Our constructive definition of reformulation in Section 4.3 is designed to lighten the demands on ATP.

4.2 Proxy Definitions and Limitations

To address the computational intractability of formulation-level verification, existing proxies consider reformulation at the instance level. For a fixed pair of instances, a proxy verifies if $\mathcal{M}'(p')$ is a reformulation of $\mathcal{M}(p)$. The simplest proxy solves $\mathcal{M}(p)$ and $\mathcal{M}'(p')$ and compares the optimal objective values [1, 25, 37]. Zhai et al. [68] propose a stronger proxy, Quasi-Karp equivalence[1], inspired by Karp reductions [30] in complexity theory.

Definition 4.4 (Quasi-Karp Equivalence [68])**.** Let $\mathcal{M}(p)$ and $\mathcal{M}'(p')$ be two instantiated formulations. We say $\mathcal{M}'(p')$ is *Quasi-Karp equivalent* to $\mathcal{M}(p)$ if there exists an algorithm $\mathcal{A}(\mathcal{M}(p), \mathcal{M}'(p'))$ that produces a mapping f such that:

- If x^* is an optimal solution to $\mathcal{M}'(p')$, then $f(x^*)$ is an optimal solution to $\mathcal{M}(p)$,
- f can be computed in polynomial time, and
- $\mathcal{A}(\mathcal{M}(p), \mathcal{M}'(p'))$ runs in polynomial time for all $p \in \mathcal{P}$ and $p' \in \mathcal{P}'$.

Quasi-Karp equivalence is an instance-level analogue of Audet reformulation. Zhai et al. [68] implement this idea in EquivaMap, where an LLM proposes a mapping f and the mapping is checked on the solved instance. This mapping-based view is more expressive than execution accuracy, allowing EquivaMap to handle some objective transformations and solution mappings on a fixed instance. However, it still does not certify at the formulation level (across all instances), and its effectiveness depends on the map class [2] for f and the LLM's ability to find the map.

Pitfalls of Instance-Level Verification. The distinction between instance and formulation-level verification is critical in automated optimization modeling. A generated formulation is meant to be reused on unseen problem data. An instance-level check can validate a reformulation that is invalid at the formulation-level, resulting in errors on unseen instances. For example, EvoCut [66] uses LLMs to propose cutting planes (cuts) for MILP formulations (Appendix F.1). A valid cut for formulation $\mathcal{M}$ removes feasible points in the linear relaxation of $\mathcal{M}$ while retaining all integer-feasible points. The EvoCut method validates candidate cuts with a simple execution-based proxy. As such, the proposed cuts may be invalid on unseen instances. The proposed cut v1-EC3 illustrates this risk (Proposition F.1). This cut eliminates triangles involving the first node. This is acceptable for an instance with $n > 3$ nodes. However, consider the 3-node instance depicted in Figure 2a with feasible tour $1 \to 2 \to 3 \to 1$. The only feasible tour is eliminated by the cut.

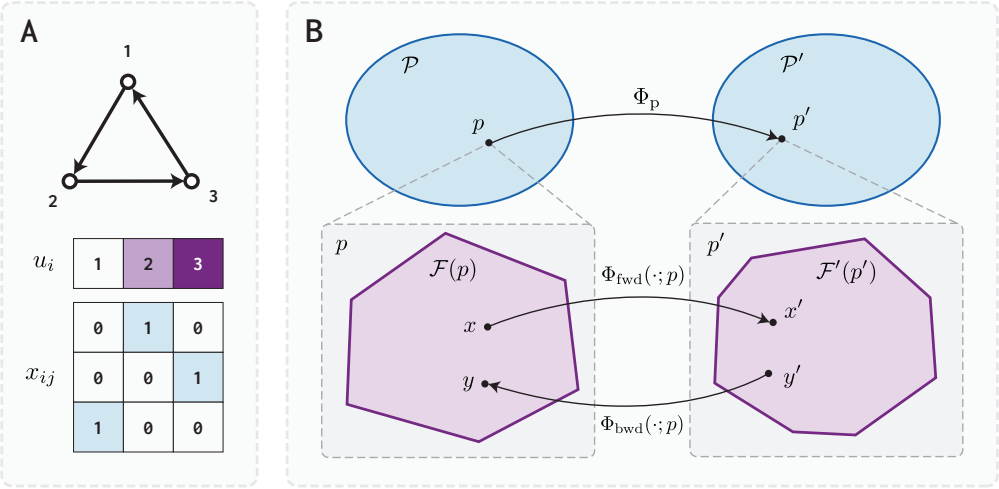

Figure 2: (a) A feasible tour $1 \to 2 \to 3 \to 1$ on a 3-node TSP instance with the MTZ variables x_{ij} and u_i. (b) A graphical depiction of the reformulation construction $\Phi(\mathcal{M}, \mathcal{M}')$. The parameter mapping Φ_{p} yields a pair of feasible regions $\mathcal{F}(p)$ and $\mathcal{F}'(p')$ related by the forward and backward maps such that objective ordering is preserved.

4.3 Constructive Definition

To overcome the pitfalls of instance-level reformulation proxies, we use formal verification to machine-check reformulations at the formulation-level. Audet reformulation can be formalized and machine-checked, but it requires ATP to prove that an optimal solution of $\mathcal{M}'(\Phi_{\mathrm{p}}(p))$ recovers an optimal solution to $\mathcal{M}(p)$. To ease the burden on ATP, we propose a constructive definition of reformulation. It strengthens Audet reformulation so that a reformulation can be verified through explicit forward and backward mappings that preserve objective ordering (see Figure 2b).

[1]Note that this definition is directional despite the *equivalence* terminology.

[2]To ensure f is computable in polynomial time, EquivaMap restricts f to be linear.

Definition 4.5 (Constructive Reformulation)**.** Let $\mathcal{M} = (\mathcal{P}, \mathcal{F}, f_0)$ and $\mathcal{M}' = (\mathcal{P}', \mathcal{F}', f_0')$ be formulations. A *reformulation construction* from $\mathcal{M}$ to $\mathcal{M}'$ is a tuple $\Phi(\mathcal{M}, \mathcal{M}') = (\Phi_{\mathrm{p}}, \Phi_{\mathrm{fwd}}, \Phi_{\mathrm{bwd}}, \Phi_{\mathrm{obj}})$ consisting of:

- a parameter mapping $\Phi_{\mathrm{p}} : \mathcal{P} \to \mathcal{P}'$,
- a forward mapping $\Phi_{\mathrm{fwd}}(\cdot; p) : \mathbb{R}^{n(p)} \to \mathbb{R}^{n'(\Phi_{\mathrm{p}}(p))}$,
- a backward mapping $\Phi_{\mathrm{bwd}}(\cdot; p) : \mathbb{R}^{n'(\Phi_{\mathrm{p}}(p))} \to \mathbb{R}^{n(p)}$ computable in polynomial time, [1] and
- an objective mapping $\Phi_{\mathrm{obj}} : \mathbb{R} \to \mathbb{R}$.

$\mathcal{M}'$ is a *constructive reformulation* of $\mathcal{M}$ if there exists a reformulation construction satisfying the following conditions for every instance $p \in \mathcal{P}$, with $p' = \Phi_{\mathrm{p}}(p)$:

- **Forward feasibility.** For all $x \in \mathcal{F}(p)$: $\Phi_{\mathrm{fwd}}(x; p) \in \mathcal{F}'(p')$.
- **Backward feasibility.** For all $x' \in \mathcal{F}'(p')$: $\Phi_{\mathrm{bwd}}(x'; p) \in \mathcal{F}(p)$.
- **Strictly monotone objective mapping.** Φ_{obj} is strictly monotonically increasing.
- **Objective preservation.** (1) For all $x \in \mathcal{F}(p)$, $f_0'(\Phi_{\mathrm{fwd}}(x; p); p') = \Phi_{\mathrm{obj}}(f_0(x; p))$, and (2) for all $x' \in \mathcal{F}'(p')$, $f_0'(x'; p') = \Phi_{\mathrm{obj}}(f_0(\Phi_{\mathrm{bwd}}(x'; p); p))$.

We say $\mathcal{M}'(p')$ is a *constructive reformulation* of $\mathcal{M}(p)$ if the conditions hold for $p \in \mathcal{P}$ and $p' \in \mathcal{P}'$.

Remark 4.6. Like Audet reformulation, this definition is only meaningful when computing the optimal solution to both formulations is NP-hard and both formulations admit a feasible point for at least one parameter.

This definition is chosen to ensure our notion of reformulation captures common MILP transformations (e.g., lifting, rescaling, and substitution) and modeling choices (e.g., exponential constraints like the TSP DFJ formulation). Our definition is similar to the definition proposed by Sherali which asserts a much stronger order-preserving bijective mapping between the two feasible regions that excludes common transformations (e.g., lifting) [38].

We prove a *constructive reformulation* is an Audet reformulation and, as an immediate corollary, the instantiated claim also implies Quasi-Karp equivalence. We defer the proof to Appendix A.

Proposition 4.7. *Let $\mathcal{M}$ and $\mathcal{M}'$ be formulations. If $\mathcal{M}'$ is a* constructive reformulation *of $\mathcal{M}$, then $\mathcal{M}'$ is an Audet reformulation of $\mathcal{M}$. Furthermore, for any instance $p \in \mathcal{P}$ with $p' = \Phi_{\mathrm{p}}(p)$, if $\mathcal{M}'(p')$ is a* constructive reformulation *of $\mathcal{M}(p)$, then $\mathcal{M}'(p')$ is* Quasi-Karp equivalent *to $\mathcal{M}(p)$.*

Our definition is stronger than Audet because it maps *all* feasible points, not only optima. This stronger notion of reformulation provides a more structured claim that reduces the proof burden on ATP while remaining satisfied by common MILP transformations. For the remainder of the paper, reformulation means *constructive reformulation*.

5 Methodology

This section operationalizes our reformulation definition into an automated verification pipeline. We first formalize our definition in Lean and then introduce FLARE, which uses ATP to generate machine-checkable certificates. Finally, we introduce FLARE-NL, a fast and cheap LLM proxy that reasons about reformulations using only natural language.

Formalization. Our reformulation definition is a formulation-level claim quantified over every instance $p \in \mathcal{P}$. To make it machine-checkable, we use a proof assistant that reasons symbolically without supplying concrete instance data. We formalize our definition in Lean [13], chosen among proof assistants for its mature Mathlib library and extensive ATP tooling (see Appendix C). We provide Lean formalizations of *formulation* (Definition 3.1) and *reformulation* (Definition 4.5) in Appendix B. We assume the input formulations are well-formed MILPs (i.e., have linear constraints and objective, and are defined over real-valued decision variables). Accordingly, our Lean formalization does not explicitly encode these conditions. The only part of Definition 4.5 we omit from the Lean proof obligation is the polynomial-time requirement on Φ_{bwd}. Formalizing computational complexity in Lean would add substantial proof burden, and in our experiments, the backward map in each construction was always computable in polynomial time.

FLARE. FLARE determines if $\mathcal{M}'$ is a reformulation of $\mathcal{M}$ under a fixed parameter mapping Φ_{p} in the following stages: (1) autoformalize $\mathcal{M}$, $\mathcal{M}'$, and Φ_{p} into Lean, (2) attempt to prove $\mathcal{M}'$ is a reformulation of $\mathcal{M}$ under Φ_{p} using ATP, and (3) check if a reformulation certificate was successfully generated by ATP. In the first two stages, we utilize the Claude Code[2] agent harness for autoformalization and ATP. Existing ATP methods, like the Numina-Lean-Agent [42],

[1] Polynomial time is measured under fixed binary encodings of parameters and rational/integer assignments: Φ_{bwd} is polynomial-time computable if a single deterministic algorithm, given p, $\Phi_{\mathrm{p}}(p)$, and x', outputs $\Phi_{\mathrm{bwd}}(x'; p)$ in time polynomial in the total input bit length. Supplying $\Phi_{\mathrm{p}}(p)$ as input means this condition does not impose a polynomial-time requirement on Φ_{p}.

[2] Alternative agent harnesses are considered in Appendix H.

demonstrate the use of Claude Code as a competitive ATP method when combined with the Lean-LSP-MCP [7, 42, 29]. We further augment Claude Code with custom agent skills for handling MILP formulations in Lean. In the final stage, we check if a proof was generated and ensure the proof compiles with no use of `sorry`[1] and no newly introduced axioms. This workflow is summarized in Figure 1 and additional implementation details can be found in Appendix C.

Limitations. FLARE's guarantee is conditional on faithful formalization. If the formulations and parameter mapping are formalized correct, then a Lean-verified certificate proves the reformulation claim under our definition. However, failure to generate a certificate is inconclusive. We highlight three limitations.

- **No certificate of invalidity.** If $\mathcal{M}'$ is not a reformulation of $\mathcal{M}$ under Φ_{p}, FLARE does not prove invalidity; the ATP component simply refuses to certify validity of the reformulation. Moreover, invalidity is scoped to the fixed mapping: it is not evidence that no parameter mapping makes $\mathcal{M}'$ a reformulation of $\mathcal{M}$.
- **Dependence on faithful formalization.** If the formalizations are unfaithful to $\mathcal{M}$, $\mathcal{M}'$, or Φ_{p}, FLARE will certify the wrong claim. We conduct an LLM-as-a-judge audit of every formalization produced during our experiments and observe no instance of this failure mode in the main FLARE experiment. Furthermore, this concern can be eliminated by deterministically translating from a modeling standard such as AMPL [18] or MathOptInterface [35] to Lean.
- **False negatives from theorem proving.** If the LLM is unable to construct an appropriate reformulation construction or the proof requires extensive work in Lean to formalize, the agent may exit early, producing a false-negative. Ongoing efforts to formalize computer science foundations in Lean with CSLib may reduce this burden on ATP.

FLARE-NL. Generating a reformulation certificate using FLARE can take 5-10 minutes. As a fast and cheap proxy, FLARE-NL prompts a frontier reasoning model with templated LaTeX descriptions of two formulations, $\mathcal{M}$ and $\mathcal{M}'$, along with the parameter mapping Φ_{p}, and asks if $\mathcal{M}'$ is a reformulation of $\mathcal{M}$ under Φ_{p} (see Appendix D for full prompt). FLARE-NL makes three changes to the naive-LLM baseline of Zhai et al. [68]: (1) the prompt includes our definition of reformulation, (2) the prompt explicitly states all formulation assumptions and instructs not to introduce new ones, and (3) we utilize reasoning and structured output. Unlike FLARE, FLARE-NL lacks any verifiable guarantees on its output. It is useful for estimating whether a certificate likely exists, but it is not a proof method.

6 Experiments

To evaluate if formulation-level verification catches failures missed by instance-level proxies, we introduce FormulationBench, an extension of the EquivaFormulation dataset [68] saturated by EquivaMap. We evaluate FLARE and FLARE-NL against established baselines and conduct an ablation study on FLARE-NL.[2] Additional experimental details are provided in Appendix G.

6.1 FormulationBench

We introduce FormulationBench, a benchmark designed to test formulation-level reformulation reasoning. FormulationBench extends EquivaFormulation [68] with EvoCut cutting plane proposals [66] and a collection of eight MILP formulation pairs provided by Ferchtandiker et al. [16]. These formulation pairs are more challenging than those in EquivaFormulation, requiring reasoning about general cutting plane families and meaningfully different modeling techniques. The dataset contains 20 problems, 109 formulations, and 89 formulation pairs with 63 positive and 26 negative examples (Appendix E). Our reformulation definition is only meaningful on NP-hard problems (Remark 4.6), so we restrict our evaluation to the subset of 16 NP-hard problems containing 54 formulation pairs. FormulationBench can be downloaded via the `formulation-bench` Python package.[3]

FormulationBench is organized similarly to the NLP4LP dataset [1]; JSON files contain formulation descriptions and problem instance data. However, FormulationBench introduces two notable extensions. We identify and record implicit assumptions necessary to prove reformulation validity. We flag assumptions implicit in the source dataset to facilitate our ablation study in Section 6.3. Second, we formalize each formulation in Lean (see Appendix B) and include ground-truth reformulation certificates for each valid reformulation pair. A handful of formulations were formalized by hand and used as reference examples to autoformalize the remainder with Claude Code. All 109 Lean formalizations were then reviewed line-by-line by a PhD student specializing in operations research, who found no autoformalization errors. While preparing this dataset, we verified prior AI-generated MILP reformulations and found 5 invalid cutting planes proposed by EvoCut [66] and 4 invalid reformulations proposed in Ferchtandiker et al. [16] (see Appendix F).

[1] The `sorry` tactic can be used in Lean proofs to skip a proof obligation. Any proof relying on `sorry` is hence incomplete.

[2] All experimental code is available at https://github.com/henryrobbins/flare

[3] Documentation is available at https://formulation-bench.henryrobbins.com

6.2 Baseline Comparison

We compare FLARE and FLARE-NL against the baselines in Zhai et al. [68]. We exclude graph-related methods due to their poor performance on non-trivial transformations [68]. Notably, the prompt provides every LLM-based method with a list of all explicit and implicit assumptions from the problem statement for use in its proof.

- **Execution [1].** Compares the optimal objective value of two formulation on a specific instance.
- **EquivaMap [68].** We re-implement EquivaMap to allow for mappings over non-scalar variables. The original mapping prompt is unmodified. We use Opus 5 as the mapping LLM.

Performance. Table 1 summarizes the main results. Our methods outperform both existing baselines. FLARE achieves **100%** accuracy and is the only method that generates machine-checkable reformulation certificates. FLARE-NL also achieves **100%** accuracy. Although it produces no certificate, FLARE-NL is **30x** faster and **25x** cheaper than FLARE. We conduct an ablation study on FLARE-NL to understand the importance of each component (Section 6.3). Both execution and EquivaMap's errors reflect limitations of instance-level validation; see Table 2 for a systematic breakdown.

Table 1: Accuracy of automated reformulation checking methods on the FormulationBench dataset. **Bold** indicates the best method on a metric and underlined indicates the second best. Metric cells report mean ± std. dev. across 3 runs; TP/FP/TN/FN reports totals. All LLM-based methods use Opus 5 with reasoning and medium effort level. Due to the high cost, we only do a single run of FLARE. FLARE and FLARE-NL outperform existing methods and FLARE is the only method that generates a machine-checkable certificate.

Method	Model	Certificate	Runs	TP	FP	TN	FN	Precision	Recall	Accuracy	Avg. Time	Avg. Cost
Execution [1]	—	✗	3	120	21	15	6	85.1%±0.0%	95.2%±0.0%	83.3%±0.0%	2.1s	—
EquivaMap [68]	Opus 5	✗	3	111	15	21	15	88.1%±0.0%	88.1%±0.0%	81.5%±0.0%	6.1s	$0.026
FLARE	Opus 5	✓	1	42	0	12	0	**100.0%**	**100.0%**	**100.0%**	410.6s	$1.180
FLARE-NL	Opus 5	✗	3	126	0	36	0	**100.0%**±0.0%	**100.0%**±0.0%	**100.0%**±0.0%	13.9s	$0.048

Perils of Instance-Level Validation. Table 2 summarizes errors by transformation type. Existing instance-level methods fail to catch formulation-level modeling errors, such as transformations (1) and (2), where a transformation can appear valid on the tested instance while failing as a general reformulation. EquivaMap's solution-mapping approach handles transformations (3) and (4), showing its advantage over the execution heuristic, but it is unable to certify validity for non-linear reformulations. In contrast, FLARE's formulation-level guarantees eliminate these false positives.

Limits of ATP. If ATP fails to produce a Lean proof, it results in a false negative. We do not oberve this failure mode in our main experiments with Opus 5. However, we run additional experiments with GPT 5.6 Sol and DeepSeek V4 Pro (see Table 6). Both models produce false negatives attributed to ATP failure. The limits of ATP are especially prominent in DeepSeek with 79.6% accuracy. Although Opus 5 achieves 100% accuracy, the cost scales with the proof difficulty due to ATP. The cost standard deviation is $1.22 and the max is $8.02. Many problems rely on standard combinatorial results that are unavailable in Lean's standard libraries (e.g., flow decomposition). As Lean libraries improve (e.g., CSLib), FLARE can reduce cost by simply invoking such results rather than reproving them.

Table 2: Accuracy of automated reformulation checking methods segmented by challenging transformation types in the FormulationBench dataset. The *Valid* column indicates if the listed transformation results in a valid reformulation. The *Pairs* column gives the number of formulation pairs in each category. Worst Case reports the minimum accuracy over the transformation categories listed above. FLARE and FLARE-NL are the only methods with **100%** accuracy in the worst case.

Transformation	Valid	Pairs	Execution [1]	EquivaMap [68]	FLARE	FLARE-NL
1. Base-10 Representation	✗	2	0%	0%	**100%**	**100%**
2. Addition of Invalid Cutting Planes	✗	3	0%	0%	**100%**	**100%**
3. Rescaled Objective	✓	2	0%	**100%**	**100%**	**100%**
4. Different Formulation (Same Objective)	✗	2	0%	**100%**	**100%**	**100%**
5. Non-Linear Solution Maps	✓	5	**100%**	0%	**100%**	**100%**
Worst Case			0%	0%	**100%**	**100%**

6.3 FLARE-NL Ablation Study

In Table 3, we ablate the FLARE-NL prompt to measure the importance of the reformulation definition and assumption handling across three LLMs. The *Baseline* uses the full prompt; *No Definition* omits the definition of reformulation; and *Allow Implicit* removes implicit assumptions from the prompt and permits the model to introduce reasonable assumptions. See Appendix D for prompt variants and Appendix G for additional details.

Explicit Definitions and Assumptions. The full FLARE-NL prompt performs the best across all model families, indicating that reformulation verification relies on explicit definitions and assumptions. Removing the reformulation definition consistently reduces accuracy, and allowing the model to reason about implicit assumptions causes the largest degradation for every model family. Verifying reformulations requires explicit criteria and assumptions, rather than model-inferred assumptions.

Frontier Model Reasoning. FLARE-NL is most effective when paired with strong frontier reasoning models (also see Appendix H). Under the full prompt, Opus 5, GPT-5.6 Sol, and DeepSeek V4 Pro all achieve high accuracy, with Opus 5 correct on every pair of every run.

Table 3: Ablation study of FLARE-NL across LLM model and prompt variations. *No Definition* omits the reformulation definition from the prompt and *Allow Implicit* removes implicit assumptions from the prompt and permits the model to introduce reasonable assumptions. Metric cells report mean ± std. dev. across 3 runs; TP/FP/TN/FN reports totals. All models have reasoning enabled and the reasoning effort level is shown following the model name.

Model (Effort)	Variant	TP	FP	TN	FN	Precision	Recall	Accuracy	Avg Time	Avg Cost
Opus 5 *(medium)*	Baseline	126	0	36	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	13.9s	$0.048
	No Definition	120	0	36	6	**100.0±0.0%**	95.2±0.0%	96.3±0.0%	12.0s	$0.039
	Allow Implicit	121	6	30	5	95.3±0.1%	96.0±1.4%	93.2±1.1%	18.4s	$0.052
GPT-5.6 Sol *(medium)*	Baseline	120	0	36	6	**100.0±0.0%**	95.2±0.0%	96.3±0.0%	14.6s	$0.034
	No Definition	120	0	36	6	**100.0±0.0%**	95.2±0.0%	96.3±0.0%	12.3s	$0.027
	Allow Implicit	93	6	30	33	93.9±0.0%	73.8±0.0%	75.9±0.0%	20.7s	$0.040
DeepSeek V4 Pro *(high)*	Baseline	118	0	36	8	**100.0±0.0%**	93.7±1.4%	95.1±1.1%	60.1s	$0.005
	No Definition	113	0	36	13	**100.0±0.0%**	89.7±1.4%	92.0±1.1%	56.9s	$0.004
	Allow Implicit	100	5	31	26	95.3±1.5%	79.4±3.6%	80.9±2.1%	71.2s	$0.005

7 Conclusion

LLMs are increasingly used for optimization modeling but lack formal guarantees, necessitating methods to automatically verify LLM-generated formulations. We develop FLARE, the first automated approach for producing machine-checkable reformulation certificates. FLARE combines a constructive definition of reformulation and ATP to verify reformulation claims at the formulation-level while existing methods only make instance-level claims. Furthermore, we introduce FLARE-NL, an LLM proxy that trades formal guarantees for improved speed and cost. Both methods outperform existing baselines and achieve **100%** accuracy on the NP-hard subset of FormulationBench.

Limitations and Future Work. To conclude, we discuss limitations of the existing FLARE methodology and motivate future work to improve formalized AI-driven optimization modeling.

- **Deterministic formalization**. The FLARE reformulation certificate requires faithful formalization. Deterministically translating accepted MILP modeling standards into Lean eliminates formalization as a source of error.
- **Reformulation definition.** FLARE's definition of reformulation is one of many. A stronger definition might be needed to certify invalidity, generally intractable under FLARE's definition. Other important relations between problems are interesting to explore formally: for example, certifying formulation strength, such as proving that one formulation yields a tighter linear relaxations. Unlike Audet reformulation, our definition rejects MILP transformations that remove strictly sub-optimal feasible points, which often generate stronger formulations. A weaker definition might be needed to assess formulation strength.
- **Real-world formulations.** The formulations of FormulationBench are drawn from academic benchmarks. Future work should extend FormulationBench to demonstrate FLARE and ATP can scale to larger, complex, industrial MILP formulations.
- **Verification for automated modeling**. Our results raise questions about how to integrate verification into LLM-driven modeling and automated algorithm design pipelines (e.g., EvoCut [66]). The combination of FLARE's machine-checkable certificates with FLARE-NL's low latency are a potentially powerful combination.

Acknowledgements

We thank the anonymous referees for thoughtful feedback that significantly improved the clarity of the paper. We also thank Nichie Supatgiat for her contributions to FormulationBench and helpful discussions. We thank Refine.ink for detailed manuscript feedback. MU gratefully acknowledges support from the Office of Naval Research under award N000142412306, Air Force Office of Scientific Research under award FA9550-26-1-0012, the Alfred P. Sloan

Foundation, the Stanford Institute for Human-Centered Artificial Intelligence (HAI), and from IBM Research as a founding member of Stanford Institute for Human-centered Artificial Intelligence. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of these funders.

References

[1] Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[2] Ali AhmadiTeshnizi, Wenzhi Gao, Herman Brunborg, Shayan Talaei, Connor Lawless, and Madeleine Udell. OptiMUS-0.3: Using Large Language Models to Model and Solve Optimization Problems at Scale. *arXiv preprint arXiv:2407.19633*, 2025.

[3] Nicolas Astorg, Tennison Liu, Yuanzhang Xiao, and Mihaela van der Schaar. Autoformulation of mathematical optimization models using LLMs. *Proceedings of Machine Learning Research*, 2025.

[4] C. Audet, P. Hansen, B. Jaumard, and G. Savard. Links Between Linear Bilevel and Mixed 0–1 Programming Problems. *Journal of Optimization Theory and Applications*, 93(2):273–300, May 1997. ISSN 1573-2878. doi: 10.1023/A:1022645805569.

[5] Axiom. From Seeing Why to Checking Everything, 2026.

[6] Egon Balas. Disjunctive programming. *Annals of discrete mathematics*, 5:3–51, 1979.

[7] Benjamin Breen, Marco Del Tredici, Jacob McCarran, Javier Aspuru Mijares, Weichen Winston Yin, Kfir Sulimany, Jacob M. Taylor, Frank H. L. Koppens, and Dirk Englund. Ax-Prover: A Deep Reasoning Agentic Framework for Theorem Proving in Mathematics and Quantum Physics. *arXiv preprint arXiv:2510.12787*, 2025.

[8] Hao Chen, Gonzalo E Constante-Flores, and Can Li. Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*, 62(4):573–587, 2024.

[9] Hao Chen, Gonzalo Esteban Constante-Flores, Krishna Sri Ipsit Mantri, Sai Madhukiran Kompalli, Akshdeep Singh Ahluwalia, and Can Li. OptiChat: Bridging optimization models and practitioners with large language models. *INFORMS Journal on Data Science*, 2025.

[10] Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Wenlei Shi, Zhihong Wang, Mingxuan Wang, Chenrui Wei, Shufa Wei, Huajian Xin, Fan Yang, Weihao Gao, Zheng Yuan, Tianyang Zhan, Zeyu Zheng, Tianxi Zhou, and Thomas Hanwen Zhu. Seed-Prover 1.5: Mastering Undergraduate-Level Theorem Proving via Learning from Experience. *arXiv preprint arXiv:2512.17260*, 2025.

[11] Michele Conforti, Gérard Cornuéjols, and Giacomo Zambelli. Integer programming models. In *Integer Programming*, pages 45–84. Springer, 2014.

[12] G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a Large-Scale Traveling-Salesman Problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954. ISSN 0096-3984.

[13] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction – CADE 28*, pages 625–635, Cham, 2021. Springer International Publishing. ISBN 978-3-030-79876-5. doi: 10.1007/978-3-030-79876-5_37.

[14] Oliver Dressler. Lean LSP MCP: Tools for agentic interaction with the lean theorem prover, March 2025.

[15] Joshua Drossman, Alexandre Jacquillat, and Sébastien Martin. Let's have a conversation: Designing and evaluating LLM agents for interactive optimization. *arXiv preprint arXiv:2604.02666*, 2026.

[16] Nathan Ferchtandiker, Dick den Hertog, Madeleine Udell, and Segev Wasserkrug. Finding efficient MILO formulations with LLMs. Working paper, 2025. URL https://github.com/nathan-ferchtandiker/LLMs-For-Optimization-Reformulations.

[17] Christodoulos A. Floudas and Xiaoxia Lin. Mixed Integer Linear Programming in Process Scheduling: Modeling, Algorithms, and Applications. *Annals of Operations Research*, 139(1):131–162, October 2005. ISSN 1572-9338. doi: 10.1007/s10479-005-3446-x.

[18] Robert Fourer, David M. Gay, and Brian W. Kernighan. *AMPL: A Modeling Language for Mathematical Programming*. Thomson/Brooks/Cole, Pacific Grove, CA, 2nd ed edition, 2003. ISBN 978-0-534-38809-6.

[19] Cameron Freer. Lean 4 Skills: Theorem proving skill and workflow pack for AI coding agents, October 2025.

[20] Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated Lean 4 dataset. In *The Thirteenth International Conference on Learning Representations*, 2025.

[21] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-completeness*. A Series of Books in the Mathematical Sciences. Freeman, New York [u.a], 27. print edition, 2009. ISBN 978-0-7167-1044-8 978-0-7167-1045-5.

[22] Bogdan Georgiev, Javier Gómez-Serrano, Terence Tao, and Adam Zsolt Wagner. Mathematical exploration and discovery at scale. *arXiv preprint arXiv:2511.02864*, 2025.

[23] Pouya Hamadanian, Pantea Karimi, Arash Nasr-Esfahany, Kimia Noorbakhsh, Joseph Chandler, Ali ParandehGheibi, Mohammad Alizadeh, and Hari Balakrishnan. Glia: A Human-Inspired AI for Automated Systems Design and Optimization. *arXiv preprint arXiv:2510.27176*, 2025.

[24] André Hottung, Federico Berto, Chuanbo Hua, Nayeli Gast Zepeda, Daniel Wetzel, Michael Römer, Haoran Ye, Davide Zago, Michael Poli, Stefano Massaroli, Jinkyoo Park, and Kevin Tierney. VRPAgent: LLM-Driven Discovery of Heuristic Operators for Vehicle Routing Problems. *arXiv preprint arXiv:2510.07073*, 2025.

[25] Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*, 73(6): 2986–3009, 2025.

[26] Logical Intelligence. Aleph Prover: State-of-the-Art Formal Theorem Prover, January 2026.

[27] Prithwish Jana, Kaan Kale, Ahmet Ege Tanriverdi, Cruise Song, Sriram Vishwanath, and Vijay Ganesh. ProofBridge: Autoformalization of natural language proofs in lean via joint embeddings. In *The Fourteenth International Conference on Learning Representations*, 2026.

[28] Caigao JIANG, Xiang Shu, Hong Qian, Xingyu Lu, JUN ZHOU, Aimin Zhou, and Yang Yu. LLMOPT: Learning to define and solve general optimization problems from scratch. In *The Thirteenth International Conference on Learning Representations*, 2025.

[29] Haocheng Ju, Guoxiong Gao, Jiedong Jiang, Bin Wu, Zeming Sun, Leheng Chen, Yutong Wang, Yuefeng Wang, Zichen Wang, Wanyi He, Peihao Wu, Liang Xiao, Ruochuan Liu, Bryan Dai, and Bin Dong. Automated Conjecture Resolution with Formal Verification. *arXiv preprint arXiv:2604.03789*, 2026.

[30] Richard M. Karp. Reducibility among Combinatorial Problems. In Raymond E. Miller, James W. Thatcher, and Jean D. Bohlinger, editors, *Complexity of Computer Computations: Proceedings of a Symposium on the Complexity of Computer Computations, Held March 20–22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, and Sponsored by the Office of Naval Research, Mathematics Program, IBM World Trade Corporation, and the IBM Research Mathematical Sciences Department*, pages 85–103. Springer US, Boston, MA, 1972. ISBN 978-1-4684-2001-2. doi: 10.1007/978-1-4684-2001-2_9.

[31] Bernard Knueven, James Ostrowski, and Jean-Paul Watson. On Mixed-Integer Programming Formulations for the Unit Commitment Problem. *INFORMS Journal on Computing*, 32(4):857–876, October 2020. ISSN 1091-9856. doi: 10.1287/ijoc.2019.0944.

[32] Wen-Yang Ku and J. Christopher Beck. Mixed Integer Programming models for job shop scheduling: A computational analysis. *Computers & Operations Research*, 73:165–173, September 2016. ISSN 0305-0548. doi: 10.1016/j.cor.2016.04.006.

[33] Connor Lawless, Jakob Schoeffer, Lindy Le, Kael Rowan, Shilad Sen, Cristina St. Hill, Jina Suh, and Bahareh Sarrafzadeh. "I Want It That Way": Enabling interactive decision support using large language models and constraint programming. *ACM Transactions on Interactive Intelligent Systems*, 14(3):1–33, 2024.

[34] Connor Lawless, Yingxi Li, Anders Wikum, Madeleine Udell, and Ellen Vitercik. LLMs for cold-start cutting plane separator configuration. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 51–69. Springer, 2025.

[35] Benoît Legat, Oscar Dowson, Joaquim Dias Garcia, and Miles Lubin. MathOptInterface: A data structure for mathematical optimization problems. *INFORMS Journal on Computing*, 2021. doi: 10.1287/ijoc.2021.1067.

[36] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large Language Models for Supply Chain Optimization. *arXiv preprint arXiv:2307.03875*, 2023.

[37] Kuo Liang, Yuhang Lu, Jianming Mao, Shuyi Sun, Chunwei Yang, Congcong Zeng, Xiao Jin, Hanzhang Qin, Ruihao Zhu, and Chung-Piaw Teo. LLM for large-scale optimization model auto-formulation: A lightweight few-shot learning approach. *arXiv preprint arXiv:2601.09635*, 2026.

[38] Leo Liberti. Reformulations in Mathematical Programming: Definitions and Systematics. *RAIRO - Operations Research*, 43 (1):55–85, January 2009. ISSN 0399-0559, 1290-3868. doi: 10.1051/ro/2009005.

[39] Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia LI, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover: A frontier model for open-source automated theorem proving. In *Second Conference on Language Modeling*, 2025.

[40] Fan Liu, Zhe-Rui Yang, Cancheng Liu, Tianrui SONG, Xiaofeng Gao, and Hao Liu. MM-Agent: LLM as agents for real-world mathematical modeling problem. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[41] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of Heuristics: Towards efficient automatic algorithm design using large language model. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[42] Junqi Liu, Zihao Zhou, Zekai Zhu, Marco Dos Santos, Weikun He, Jiawei Liu, Ran Wang, Yunzhou Xie, Junqiao Zhao, Qiufeng Wang, Lihong Zhi, Jia Li, and Wenda Li. Numina-Lean-Agent: An Open and General Agentic Reasoning System for Formal Mathematics. *arXiv preprint arXiv:2601.14027*, 2026.

[43] Hugues Marchand, Alexander Martin, Robert Weismantel, and Laurence Wolsey. Cutting planes in integer and mixed integer programming. *Discrete Applied Mathematics*, 123(1):397–446, November 2002. ISSN 0166-218X. doi: 10.1016/S0166-218X(01)00348-1.

[44] C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer Programming Formulation of Traveling Salesman Problems. *J. ACM*, 7 (4):326–329, October 1960. ISSN 0004-5411. doi: 10.1145/321043.321046.

[45] Hugo Morais, Péter Kádár, Pedro Faria, Zita A. Vale, and H. M. Khodr. Optimal scheduling of a renewable micro-grid in an isolated load area using mixed-integer linear programming. *Renewable Energy*, 35(1):151–156, January 2010. ISSN 0960-1481. doi: 10.1016/j.renene.2009.02.031.

[46] Mahdi Mostajabdaveh, Timothy T Yu, Rindranirina Ramamonjison, Giuseppe Carenini, Zirui Zhou, and Yong Zhang. Optimization modeling and verification from problem specifications using a multi-agent multi-stage LLM framework. *INFOR: Information Systems and Operational Research*, 62(4):599–617, 2024.

[47] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[48] Yves Pochet and Laurence A Wolsey. *Production Planning by Mixed Integer Programming*. Springer, 2006.

[49] Rindranirina Ramamonjison, Timothy Yu, Raymond Li, Haley Li, Giuseppe Carenini, Bissan Ghaddar, Shiqi He, Mahdi Mostajabdaveh, Amin Banitalebi-Dehkordi, Zirui Zhou, et al. NL4Opt competition: Formulating optimization problems based on their natural language descriptions. In *NeurIPS 2022 Competition Track*, pages 189–203. PMLR, 2023.

[50] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

[51] Borja Requena, Austin Letson, Krystian Nowakowski, Izan Beltran Ferreiro, and Leopoldo Sarra. A Minimal Agent for Automated Theorem Proving. *arXiv preprint arXiv:2602.24273*, 2026.

[52] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, January 2024. ISSN 0028-0836, 1476-4687. doi: 10.1038/s41586-023-06924-6.

[53] K. Srinivasan, K.S. Chatha, and G. Konjevod. Linear-programming-based techniques for synthesis of network-on-chip architectures. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 14(4):407–420, April 2006. ISSN 1557-9999. doi: 10.1109/TVLSI.2006.871762.

[54] Zachary Steever, Kyle Hunt, Mark Karwan, Junsong Yuan, and Chase C Murray. A graph-based approach for relating integer programs. *INFORMS Journal on Computing*, 36(6):1715–1736, 2024.

[55] Tony J Van Roy and Laurence A Wolsey. Solving mixed integer programming problems using automatic reformulation. *Operations Research*, 35(1):45–57, 1987.

[56] François Vanderbeck and Laurence A. Wolsey. Reformulation and Decomposition of Integer Programs. In Michael Jünger, Thomas M. Liebling, Denis Naddef, George L. Nemhauser, William R. Pulleyblank, Gerhard Reinelt, Giovanni Rinaldi, and Laurence A. Wolsey, editors, *50 Years of Integer Programming 1958-2008*, pages 431–502. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010. ISBN 978-3-540-68274-5 978-3-540-68279-0. doi: 10.1007/978-3-540-68279-0_13.

[57] Sumanth Varambally, Thomas Voice, Yanchao Sun, Zhifeng Chen, Rose Yu, and Ke Ye. Hilbert: Recursively building formal proofs with informal reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026.

[58] Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, Jiawei Liu, Jonas Bayer, Julien Michel, Longhui Yu, Léo Dreyfus-Schmidt, Lewis Tunstall, Luigi Pagani, Moreira Machado, Pauline Bourigault, Ran Wang, Stanislas Polu, Thibaut Barroyer, Wen-Ding Li, Yazhe Niu, Yann Fleureau, Yangyang Hu, Zhouliang Yu, Zihan Wang, Zhilin Yang, Zhengying Liu, and Jia Li. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning. *arXiv preprint arXiv:2504.11354*, 2025.

[59] Zhuohan Wang, Ziwei Zhu, Yizhou Han, Yufeng Lin, Zhihang Lin, Ruoyu Sun, and Tian Ding. OptiBench: Benchmarking large language models in optimization modeling with equivalence-detection evaluation, 2024.

[60] Zhuohan Wang, Ziwei Zhu, Ziniu Li, Congliang Chen, Yizhou Han, Yufeng Lin, Zhihang Lin, Angyang Gu, Xinglin Hu, Ruoyu Sun, et al. ORGEval: Graph-theoretic evaluation of LLMs in optimization modeling. *arXiv preprint arXiv:2510.27610*, 2025.

[61] Laurence A Wolsey. *Integer Programming*. John Wiley & Sons, 2020.

[62] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

[63] Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, and Gang Chen. Chain-of-Experts: When LLMs Meet Complex Operations Research Problems. In *The Twelfth International Conference on Learning Representations*, 2024.

[64] Zhuoliang Xie, Fei Liu, Zhenkun Wang, and Qingfu Zhang. Enhancing CVRP Solver through LLM-driven Automatic Heuristic Design. *arXiv preprint arXiv:2602.23092*, 2026.

[65] Linzi Xing, Xinglu Wang, Yuxi Feng, Zhenan Fan, Jing Xiong, Zhijiang Guo, Xiaojin Fu, Rindra Ramamonjison, Mahdi Mostajabdaveh, Xiongwei Han, Zirui Zhou, and Yong Zhang. Towards Human-aligned Evaluation for Linear Programming Word Problems. In Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue, editors, *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 16550–16556, Torino, Italia, May 2024. ELRA and ICCL.

[66] Milad Yazdani, Mahdi Mostajabdaveh, Samin Aref, and Zirui Zhou. EvoCut: Strengthening Integer Programs via Evolution-Guided Language Models. *arXiv preprint arXiv:2508.11850*, 2025.

[67] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[68] Haotian Zhai, Connor Lawless, Ellen Vitercik, and Liu Leqi. EquivaMap: Leveraging LLMs for automatic equivalence checking of optimization formulations. In *Forty-second International Conference on Machine Learning*, 2025.

A Proof of Proposition 4.7

Proposition 4.7. *Let $\mathcal{M}$ and $\mathcal{M}'$ be formulations. If $\mathcal{M}'$ is a* constructive reformulation *of $\mathcal{M}$, then $\mathcal{M}'$ is an Audet reformulation of $\mathcal{M}$. Furthermore, for any instance $p \in \mathcal{P}$ with $p' = \Phi_{\mathrm{p}}(p)$, if $\mathcal{M}'(p')$ is a* constructive reformulation *of $\mathcal{M}(p)$, then $\mathcal{M}'(p')$ is* Quasi-Karp equivalent *to $\mathcal{M}(p)$.*

Proof. Let $\mathcal{M} = (\mathcal{P}, \mathcal{F}, f_0)$ and $\mathcal{M}' = (\mathcal{P}', \mathcal{F}', f_0')$ be formulations. Assume $\mathcal{M}'$ is a constructive reformulation of $\mathcal{M}$. By definition, there exists a reformulation construction $\Phi(\mathcal{M}, \mathcal{M}') = (\Phi_{\mathrm{p}}, \Phi_{\mathrm{fwd}}, \Phi_{\mathrm{bwd}}, \Phi_{\mathrm{obj}})$ such that the conditions of Definition 4.5 hold. Fix an instance $p \in \mathcal{P}$ and let $p' = \Phi_p(p)$.

We first show that if $\mathcal{M}(p)$ has an optimal solution, then $\mathcal{M}'(p')$ also has an optimal solution. Let $x^* \in \mathcal{F}(p)$ be optimal for $\mathcal{M}(p)$. By forward feasibility, $x' = \Phi_{\mathrm{fwd}}(x^*; p) \in \mathcal{F}'(p')$. We now claim that x' is optimal for $\mathcal{M}'(p')$. Suppose this was not true. There must exist an $\hat{x}' \in \mathcal{F}'(p')$ such that $f_0'(\hat{x}') < f_0'(x')$. By backward feasibility, $\hat{x} = \Phi_{\mathrm{bwd}}(\hat{x}'; p) \in \mathcal{F}(p)$. By objective preservation:

$$f_0'(\hat{x}'; p') = \Phi_{\mathrm{obj}}(f_0(\hat{x}; p)) \quad \text{and} \quad f_0'(x'; p') = \Phi_{\mathrm{obj}}(f_0(x^*; p)).$$

Thus,

$$\Phi_{\mathrm{obj}}(f_0(\hat{x}; p)) < \Phi_{\mathrm{obj}}(f_0(x^*; p)).$$

Since Φ_{obj} is strictly increasing, this implies $f_0(\hat{x}; p) < f_0(x^*; p)$ which contradicts the optimality of x^*. Using an identical argument by contradiction, we can show that every optimal solution of $\mathcal{M}'(p')$ is mapped to an optimal solution of $\mathcal{M}(p)$ by the backward mapping $\Phi_{\mathrm{bwd}}(\cdot; p)$. Definition 4.5 requires $\Phi_{\mathrm{bwd}}(\cdot; p)$ to be computable in polynomial time. Therefore, given any optimal solution of $\mathcal{M}'(p')$, an optimal solution of $\mathcal{M}(p)$ can be recovered in polynomial time. Hence $\mathcal{M}'$ is an Audet reformulation of $\mathcal{M}$.

For the instantiated claim, fix p and $p' = \Phi_p(p)$. Definition 4.4 requires a mapping-producing algorithm $\mathcal{A}$. Let $\mathcal{A}(\mathcal{M}(p), \mathcal{M}'(p'))$ output the backward mapping Φ_{bwd} with p bound as a constant, representing the mapping

$$f(x') = \Phi_{\mathrm{bwd}}(x'; p).$$

This only copies p into a fixed-size algorithm, so $\mathcal{A}$ runs in polynomial time for all $p \in \mathcal{P}$ and $p' \in \mathcal{P}'$. By Definition 4.5, f is polynomial-time computable, and the argument above shows f maps every optimal solution of $\mathcal{M}'(p')$ to an optimal solution of $\mathcal{M}(p)$. Thus $\mathcal{M}'(p')$ is Quasi-Karp equivalent to $\mathcal{M}(p)$. □

B Lean Formalization

We now formalize the *formulation* (Definition 3.1) and *reformulation* (Definition 4.5) definitions in Lean. For both definitions, we utilize a Lean `structure`. A `structure` is a collection of named fields and their types. See Listing 1 for the `MILPFormulation` and `MILPReformulation` structures. In the following sections, we provide details on how each definition is formalized.

B.1 Formulation

A formulation $\mathcal{M} = (\mathcal{P}, \mathcal{F}, f_0)$ is represented by the `MILPFormulation` structure. The `Params` field encodes the parameter space $\mathcal{P}$. Next, the `Vars` and `feasible` fields together encode the feasible region $\mathcal{F}$. Lastly, `obj` encodes the objective function f_0. Notice that both `feasible` and `obj` are functions of `Params` and `Vars`. See Listing 2 for an example Lean formalization of the TSP MTZ formulation.

Discrepancies. This definition does not restrict `Vars` to $\mathbb{R}^n$ nor does it assert linearity conditions on either `feasible` or `obj`. This additional typing would add modeling complexity to `Vars`, reduce legibility, and place unnecessary burden on ATP. Our primary motivation is not to prove if a MILP formulation is well-formed. Hence, we make the practical decision to exclude this typing.

B.2 Reformulation

We formalize reformulation with the `MILPReformulation` structure. This defines the reformulation construction $\Phi(\mathcal{M}, \mathcal{M}') = (\Phi_{\mathrm{p}}, \Phi_{\mathrm{fwd}}, \Phi_{\mathrm{bwd}}, \Phi_{\mathrm{obj}})$ along with the four conditions specified in Definition 4.5. The fields `paramMap`, `fwd`, `bwd`, and `obj` encode Φ_{p}, Φ_{fwd}, Φ_{bwd}, and Φ_{obj} respectively. The fields `fwd_feas` and `bwd_feas` encode the forward and backward feasibility conditions. The two objective mapping conditions are encoded by `fwd_obj` and `bwd_obj`. Lastly, `objMap_mono` encodes the strict monotonicity objective condition. A formulation `G` is proven to be a reformulation of `F` by declaring a definition of type `MILPReformulation F G`. This structure encodes both the

witnessing reformulation construction and proves it obeys the necessary conditions. Proving `G` is *not* a reformulation of `F` requires proving the type `MILPReformulation F G` is uninhabited (i.e., no construction satisfying the conditions exists).

Discrepancies. The Lean formalization does not enforce that the backward mapping Φ_{bwd} is computable in polynomial time. Formalizing computational complexity in Lean is non-trivial and adds substantial proof burden to ATP. In our experiments, we observe no examples of LLMs producing non-polynomial-time backward maps, so we om it the condition from the Lean proof obligation.

Listing 1: Lean formalizations: `MILPFormulation` and `MILPReformulation`.

```
import Mathlib.Tactic
import Mathlib.Data.Real.Basic
import Mathlib.Order.Basic

structure MILPFormulation where
  Params   : Type                                    -- Parameter space
  Vars     : Params → Type                           -- Variables
  feasible : (p : Params) → Vars p → Prop            -- Feasible region
  obj      : (p : Params) → Vars p → ℝ               -- Objective function

structure MILPReformulation (F G : MILPFormulation) where
  paramMap    : F.Params → G.Params                  -- Parameter mapping
  fwd         : (p : F.Params) → F.Vars p →          -- Forward mapping
                G.Vars (paramMap p)
  bwd         : (p : F.Params) → G.Vars (paramMap p) → -- Backward mapping
                F.Vars p
  fwd_feas    : ∀ p x, F.feasible p x →              -- Forward feasibility condition
                G.feasible (paramMap p) (fwd p x)
  bwd_feas    : ∀ p x', G.feasible (paramMap p) x' →  -- Backward feasibility condition
                F.feasible p (bwd p x')
  objMap      : ℝ → ℝ                                -- Objective mapping
  objMap_mono : StrictMono objMap                    -- Objective monotonicity
  fwd_obj     : ∀ p x, F.feasible p x →              -- Forward objective condition
                G.obj (paramMap p) (fwd p x) = objMap (F.obj p x)
  bwd_obj     : ∀ p x', G.feasible (paramMap p) x' →  -- Backward objective condition
                G.obj (paramMap p) x' = objMap (F.obj p (bwd p x'))
```

Listing 2: An example Lean formalization of the TSP MTZ formulation.

```
structure Params where
  n : ℕ                          -- number of cities
  c : Fin n → Fin n → ℝ          -- arc cost
  hn : 2 ≤ n

structure Vars (p : Params) where
  x : Fin p.n → Fin p.n → ℤ      -- arc indicator
  u : Fin p.n → ℝ                -- position

structure Feasible (p : Params) (v : Vars p) : Prop where
  -- Each city has exactly one outgoing arc
  hout : ∀ i : Fin p.n, ∑ j : Fin p.n, v.x i j = 1
  -- Each city has exactly one incoming arc
  hin : ∀ j : Fin p.n, ∑ i : Fin p.n, v.x i j = 1
  -- MTZ subtour elimination
  hmtz : ∀ (i : Fin p.n) (j : Fin p.n), i.val ≠ 0 → j.val ≠ 0 → i ≠ j →
    v.u i - v.u j + (p.n : ℝ) * (v.x i j : ℝ) ≤ (p.n : ℝ) - 1
  -- Depot position fixed to 1
  hu_depot : haveI : NeZero p.n := <by have := p.hn; omega>; v.u 0 = 1
  hx_bin : ∀ (i j : Fin p.n), v.x i j = 0 ∨ v.x i j = 1
  -- u ∈ [2, n] for non-depot cities
  hu_lo : ∀ i : Fin p.n, i.val ≠ 0 → 2 ≤ v.u i
  hu_hi : ∀ i : Fin p.n, v.u i ≤ (p.n : ℝ)
  -- No self-loops
  hx_no_self : ∀ i : Fin p.n, v.x i i = 0

-- Minimize total arc cost
def obj (p : Params) (v : Vars p) : ℝ :=
  ∑ i : Fin p.n, ∑ j : Fin p.n, p.c i j * (v.x i j : ℝ)
```

C FLARE Implementation Details

This section outlines the implementation details for every component of FLARE (Figure 1).[1] We use a general-purpose coding agent harness (Claude Code, Codex, and OpenCode) for both autoformalizing MILP formulations and ATP. This design decision was inspired by recent work achieving competitive ATP performance with Claude Code [42]. In Section C.1, we describe the agent harness prompt, working directory, and sandbox. To enable the agent to effectively compose Lean proofs, we use the Lean-LSP-MCP (Section C.2) and two custom agent skills (Section C.3). After the agent exits, FLARE does a final verification to check if a reformulation certificate was successfully generated (Section C.4).

C.1 Agent Harness

We programmatically initialize the agent harness in a headless mode using the CLI. The agent is given instructions and provided with a working directory containing the necessary context. To isolate the agent's working directory and avoid duplicating the Lean environment (the Mathlib library is over 5GB), we run FLARE inside a Docker container.

Agent Prompt. In the agent prompt (Listing 3), we provide the agent with instructions to (1) formalize both MILP formulations as `MILPFormulation` structures `A` and `B` and (2) attempt to declare a definition of type `MILPReformulation A B` where `paramMap` formalizes the fixed parameter map.

Working Directory Files. The working directory is initialized with the following context:

- templated LaTeXdescriptions of both MILP formulations (Listing 5) and the parameter map (Listing 6),
- Gurobi Python implementations for both MILP formulations and the parameter mapping, and
- a Lean file `Common.lean` with `MILPFormulation` and `MILPReformulation` definitions.

Both the templated LaTeX descriptions and Gurobi Python implementations are populated by the FormulationBench JSON files. The `formulation-bench` Python package provides utilities to construct Gurobi Python implementations from the JSON file code snippets. See Appendix E for additional details on the FormulationBench dataset structure.

Docker. We pre-build a Docker image called `flare-agent` with every agent CLI (Claude Code, Codex, and OpenCode), the Lean-LSP-MCP, and a Lake-built Lean environment with Mathlib pre-compiled. When FLARE is invoked, it creates a working directory on the host with all the necessary files. It then creates a Docker container from this image and copies the working directory into the container. To use the pre-built Lean environment, we symlink `.lake` from the pre-built location into the agent's working directory. This configuration prevents duplication of the Lean environment while ensuring FLARE can run in parallel without agents impacting each other's Lean environment. Previously, we attempted to isolate agents with the permissions and sandboxing mechanisms provided by each agent harness. We found the implementations to be immature; Docker was the only reliable way to ensure the agent couldn't access files outside its working directory.

C.2 Lean-LSP-MCP

Lean-LSP-MCP [14][2] is a Model Context Protocol (MCP) server providing MCP Tools for Lean theorem proving. It enables the agent to efficiently generate, debug, and compile Lean proofs and has been utilized by numerous Lean ATP methods [7, 42, 29]. The Lean Language Server Protocol (LSP) traditionally allows editors like VS Code to get rich, interactive feedback from a running Lean process. The Lean-LSP-MCP allows the agent to communicate directly with this LSP and obtain feedback like a human theorem prover. It offers a broad range of tools including:

- `lean_goal`. Get the proof goal at a specific location in the file. This allows the agent to observe precisely what sub-goal must be proved to continue making progress.
- `lean_diagnostic_messages`. Retrieve all the diagnostic messages for a Lean file. This allows the agent to verify if the file is compiling and, if not, where fixes are necessary.
- `lean_verify`. Verify the soundness of a proof. There are two conditions where a proof compiles, but is actually unsound: (1) a new `axiom` was added or (2) the `sorry` tactic is used to complete a goal (see Appendix C.4 for a further discussion). This tool allows the agent to verify that neither condition holds.
- `lean_multi_attempt`. Attempt multiple tactics at a proof position and return the new goal state for each. This allows the agent to explore strategies for making progress in parallel.
- `lean_hover_info`. Retrieve documentation for symbols, terms, and expressions. This allows the agent to view underlying definitions and reduces hallucination.

[1] FLARE is implemented in the `milp-flare` Python package; see https://flare.henryrobbins.com.

[2] https://github.com/oOo0oOo/lean-lsp-mcp

C.3 Agent Skills

Agent skills are an "open format for extending AI agent capabilities with specialized knowledge and workflows." They are simply a directory containing context, scripts, or other resources related to a specific task. The directory contains a `SKILL.md` file which tells the agent the skill's name and provides instructions on when to use it. Skills allow for *progressive disclosure* where the agent loads additional task-specific context as needed.

We define two custom agent skills, `lean-milp-formulation` and `lean-milp-reformulation`, for autoformalizing MILP formulations and proving reformulations respectively. The agent prompt explicitly instructs the agent to invoke these skills. Both skills contain instructions along with a template Lean file containing detailed comments about Lean modeling choices and file structure. In addition to our custom skills, we use the general-purpose Lean 4 skills [19].[1]

C.4 Final Verification

After the agent exits, we inspect the working directory. First, we verify the presence of `A/Formulation.lean`, `B/Formulation.lean`, and `Reformulation.lean`. If all of these files are present and compile, we then check that `Reformulation.lean` contains a `MILPReformulation A B` definition. Finally, we guard against two conditions where a proof compiles but is unsound: use of the `sorry` tactic and the introduction of non-standard axioms. The `sorry` tactic can skip any proof obligation and indicates the proof has yet to be formally verified. We emit `#print axioms` for the `MILPReformulation A B` definition, which reports its transitive axiom dependencies; a `sorry` surfaces here as `sorryAx`. We require that the proof depend only on Lean's three standard axioms (`propext`, `Classical.choice`, and `Quot.sound`). If all files compile and the `MILPReformulation A B` definition is `sorry`-free and introduces no new axioms, formulation `B` is deemed a reformulation of `A` under the fixed parameter mapping.

[1] https://github.com/cameronfreer/lean4-skills

D Prompts

Listing 3: FLARE Agent Prompt

```
You are a mixed-integer linear programming (MILP) and Lean 4 expert. You have been tasked with formalizing two MILP
      formulations in Lean 4 and then proving that formulation B is a reformulation of formulation A.

## Working Directory Structure

The working directory will be initialized with the following structure:

'''
  +-- A
  |   +-- Formulation.lean    # Write the Lean 4 formalization of Formulation A here
  |   +-- formulation.md      # Natural language description of Formulation A
  |   \-- solve.py            # Python script that solves Formulation A
  +-- B
  |   +-- Formulation.lean    # Write the Lean 4 formalization of Formulation B here
  |   +-- formulation.md      # Natural language description of Formulation B
  |   \-- solve.py            # Python script that solves Formulation B
  +-- Common.lean             # Definition of 'MILPFormulation' and 'MILPReformulation'
  +-- Reformulation.lean      # Write the reformulation proof here
  +-- map.md                  # The parameter mapping from Formulation A to Formulation B
  +-- map.py                  # Python script computing B's parameters from A's
  +-- lake-manifest.json      # DO NOT EDIT!
  +-- lakefile.toml           # DO NOT EDIT!
  \-- lean-toolchain          # DO NOT EDIT!
'''

## Parameter Map
The parameter mapping of the reformulation construction is *given*, not something you search for. 'map.md' states, for each
      parameter of Formulation B, how it is computed from the parameters of Formulation A; 'map.py' is the same mapping as an
      executable Python script. Read 'map.md' before writing either 'Formulation.lean' file.

## Workflow
1. Read 'map.md' (and 'map.py' where the LaTeX is ambiguous) to understand the given parameter mapping.
2. Utilize the 'lean-milp-formulation' skill to generate both 'Formulation.lean' files.
3. Validate each 'Formulation.lean' file with 'mcp__lean-lsp__lean_diagnostic_messages'. Fix any issues that arise and repeat
      until both files compile cleanly.
4. Run 'Bash(lake build A.Formulation B.Formulation)' to materialize their oleans.
5. Determine whether formulation B is a mathematical reformulation of formulation A *under the given parameter mapping*.
6. If B is a reformulation of A, use the 'lean-milp-reformulation' skill to generate a compiled 'MILPReformulation' proof in '
      Reformulation.lean', with its 'paramMap' field implementing the mapping in 'map.md'.
7. Validate 'Reformulation.lean' with 'mcp__lean-lsp__lean_verify' to ensure there are no stubs. Fix any issues that arise and
      repeat until both files compile cleanly.

## Rules
- IMPORTANT: Only read/write files that exist in *this* working directory. Do not navigate outside of it for any reason.
- The 'paramMap' field of your 'MILPReformulation' MUST implement the mapping given in 'map.md'. Do not substitute a different
      parameter mapping, even if another one would make the proof easier. If B is not a reformulation of A under *this* mapping,
       that is a negative result -- report it as described below rather than switching mappings.
- 'map.md' also constrains how you formalize each 'Formulation.lean': B's 'Params' structure must have exactly the parameters
      that 'map.md' computes, and A's 'Params' must have the parameters 'map.md' computes them from. If you cannot express the
      mapping as a total function 'A.Params -> B.Params', the mismatch is in your formalization -- fix the 'Params' structures
      rather than adapting the mapping.
- DO NOT EDIT 'map.md' or 'map.py'.
- The Lean project root is the current directory. Use 'import A.Formulation' and 'import B.Formulation'.
- You MUST use the lean-lsp MCP tools (mcp__lean-lsp__*) to check compilation as you work. Before doing anything else, verify
      the server is available by calling 'mcp__lean-lsp__lean_diagnostic_messages' on 'Common.lean'. If the first probe reports
      the tool as unregistered or still connecting, that is expected -- probe again until it answers, up to 3 attempts. If the *
      same* server-level failure (e.g. "Failed to start Lean language server", tool not registered) persists, conclude the
      environment is unusable: write 'MCP_UNAVAILABLE: <error>' to Reformulation.lean and exit. Do not fall back to 'lake env
      lean' to perform compilation.
- If 'lake build' fails, treat the error message as a real signal -- read it, fix the cause (typically a typo, missing import,
      or stale olean), and retry. Only after the *same* failure persists across two clean retries should you write '
      LAKE_BUILD_FAILED: <error>' to Reformulation.lean and exit. Persistent toolchain-level failures (e.g. elan/toolchain
      missing) are the exit case; ordinary compile errors are not.
- Generate both Formulation.lean files before attempting the reformulation proof.
- Confirm the final reformulation proof with 'mcp__lean-lsp__lean_verify' on the 'MILPReformulation' definition. The returned
      axioms must NOT contain 'sorryAx' -- if it does, the proof has a stub and you must finish it.
- You are expected to iterate on the proof until it compiles and 'lean_verify' reports no 'sorryAx'. A non-trivial
      reformulation proof typically runs 100+ lines with many manual 'refine'/'rcases'/'simp' steps and multiple rounds of
      compile-error fixing. You should only exit before this point if there is concrete evidence that B is *not* a reformulation
       of A under the given assumptions.

## Common Mistakes
- Interpret 'lean_diagnostic_messages' carefully: 'success:true, items:[]' means the file compiles cleanly. 'success:false,
      items:[]' typically means imports aren't built yet -- build them, don't assume the file is broken. Real errors come back
      as 'items' with severity/message fields.
```

```
- If there issues with the Lean environment or MCP tools, it is imperative to handle them as described in the rules above.
      Report and exit.
- If you are stuck proving the reformulation due missing assumptions in the ‘Formulation.lean‘ files, first verify that the
      assumptions specified in ‘formulation.md‘ are all present in the corresponding ‘Formulation.lean‘. If any assumptions are
      missing, add them. Otherwise, DO NOT ADD ASSUMPTIONS YOURSELF. Instead, report the missing necessary assumptions as an
      issue preventing the reformulation proof in ‘Reformulation.lean‘ and exit.

## Available Tools
**Filesystem:** ‘Bash(ls ./*)‘, ‘Bash(find ./*)‘,
**Read:** ‘Read(./*)‘, ‘Bash(cat ./*)‘, ‘Bash(head ./*)‘, ‘Bash(tail ./*)‘, ‘Bash(less ./*)‘, ‘Bash(more ./*)‘, ‘Bash(bat ./*)‘
**Edit:** ‘Edit(./*)‘
**Write:** ‘Write(./*)‘
**Skills:** ‘lean4:lean4‘, ‘lean-milp-formulation‘, ‘lean-milp-reformulation‘
**MCP Tools:** ‘mcp__lean-lsp__*‘
**Lake:** ‘Bash(lake env lean:*)‘, ‘Bash(lake build:*)‘
```

Listing 4: FLARE-NL Prompt. The *No Definition* variant omits L9-27 and drops the Φ_p symbol from L31. The *Allow Implicit* variant omits L38 as well as implicit assumptions and constraints from the formulation descriptions. The formulation descriptions are populated by Listing 5 and the parameter mapping is populated by Listing 6.

```
1  You are given the following two Mixed-Integer Linear Programming (MILP) formulations. You are tasked with deciding if
       formulation B is a reformulation of formulation A.
2
3  ## Formulations
4
5  {{ formulation_a }}
6
7  {{ formulation_b }}
8
9  ## Definitions
10
11 Use the following definition of formulation and reformulation to guide your reasoning.
12
13 **Formulation.** A MILP *formulation* $A$ is a tuple ${A = (P, F, f_0)}$ with parameter space $P$, feasible region $F(p) \
       subseteq R^{n(p)}$, and objective function $f_0$. For instance $p \in P$, the feasible region $F(p)$ is defined by $m(p)
       $ linear constraints, $f_i(\cdot ; p) : R^{n(p)} \to R$ for all $i \in [m(p)]$. The first $k(p) \leq n(p)$ variables are
        integers. The feasible region is
14 $$F(p) = \{x \in \mathbb{Z}^{k(p)} \times R^{n(p)-k(p)}~|~f_i(x;p) \leq 0 ~\forall i\in[m(p)]\}.$$
15 The objective is to minimize the linear function $f_0(\cdot ; p) : R^{n(p)} \to R$. A formulation $A$ is *instantiated* with
       an *instance* $p \in P$. We denote an instantiated formulation as ${A(p) = (F(p), f_0(p))}$.
16
17 **Reformulation.** Let $A = (P, F, f_0)$ and $B = (P', F', f'_0)$ be formulations. A *reformulation construction* from $A$ to
        $B$ is a tuple $\Phi(A, B) = (\Phi_{p},\, \Phi_{fwd},\, \Phi_{bwd},\, \Phi_{\text{obj}})$ consisting of:
18 - a parameter mapping $\Phi_{p} : P \to P'$,
19 - a forward mapping $\Phi_{fwd}(\cdot;p) : R^{n(p)} \to R^{n'(\Phi_{p}(p))}$,
20 - a backward mapping $\Phi_{bwd}(\cdot;p) : R^{n'(\Phi_{p}(p))} \to R^{n(p)}$ computable in polynomial time, and
21 - an objective mapping $\Phi_{\text{obj}} : R \to R$.
22
23 $B$ is a *reformulation* of $A$ if there exists a reformulation construction satisfying the following conditions for every
       instance $p \in P$, with $p' = \Phi_{p}(p)$:
24 - **Forward feasibility.** For all feasible points $x \in F(p)$, the forward mapping maps to a feasible point $x' = \Phi_{fwd
       }(x;p) \in F'(\Phi_{p}(p))$.
25 - **Backward feasibility.** For all feasible points $x' \in F'(p')$, the backward mapping maps to a feasible point $x = \Phi_{
       bwd}(x';p) \in F(p)$.
26 - **Objective mapping.** The forward and backward mappings induce an objective mapping. The following two conditions must
       hold. (1) For all feasible points ${x \in F(p)}$, the forward mapped point $x' = \Phi_{fwd}(x;p)$ has objective value
       $f_0'(x';p') = \Phi_{\text{obj}}(f_0(x;p))$. (2) For all feasible points ${x' \in F'(p')}$, the backward mapped point is
        $x = \Phi_{bwd}(x';p)$ and $f_0'(x';p') = \Phi_{\text{obj}}(f_0(x;p))$.
27 - **Strictly monotone.** The objective mapping $\Phi_{\text{obj}}$ is strictly monotonically increasing.
28
29 ## Parameter Mapping
30
31 The parameter mapping $\Phi_{p}$ is *given*. It computes each parameter of formulation B from the parameters of formulation A.


32
33 {{ parameter_map }}
34
35 ## Instructions
36
37 - Decide whether B is a reformulation of A under *this* parameter mapping. Do not substitute a different one, even if another
        mapping would make B a reformulation of A.
38 - Do NOT make any assumptions about the formulation or parameter space that are not explicitly stated in the formulation
       descriptions.
39 - When uncertain, state that formulation B is *not* a reformulation of A.
40 - Provide a short summary of your conclusion (at most 2,500 characters) and a final determination of whether B is a
       reformulation of A (true or false).
```

Listing 5: Formulation Prompt Template. Rendered as `formulation.md` for FLARE and `{{ formulation_(a|b) }}` in the FLARE-NL prompt.

```
# {{ problem_name }}

## Problem Description

{{ problem_description }}

## Formulation

### Parameters

{% for name, p in parameters.items() %}
- **{{ name }}** (type: {{ p.type.value }}, shape: '{{ p.shape }}'): {{ p.description }}
{% endfor %}

{% if assumptions %}

### Assumptions

{% for a in assumptions %}
- {{ a.description }}
$${{ a.formulation }}$$
{% endfor %}
{% endif %}

{% if definitions %}

### Definitions

{% for name, d in definitions.items() %}
- **{{ name }}**: {{ d.description }}
$${{ d.formulation }}$$
{% endfor %}
{% endif %}

### Variables

{% for name, v in variables.items() %}
- **{{ name }}** (type: {{ v.type.value }}, shape: '{{ v.shape }}'): {{ v.description }}
{% endfor %}

### Constraints

{% for c in constraints %}
- {{ c.description }}
$${{ c.formulation }}$$
{% endfor %}

### Objective

{{ objective.description }}
$${{ objective.formulation }}$$
```

Listing 6: Parameter Map Template. Rendered as `map.md` for FLARE and `{{ parameter_map }}` in the FLARE-NL prompt.

```
# Parameter Map

{% for note in notes %}
{{ note }}

{% endfor %}
{% if definitions %}
## Definitions

{% for name, d in definitions.items() %}
- **{{ name }}**
$${{ d.formulation }}$$
{% endfor %}

{% endif %}
## Parameters

{% for name, d in parameters.items() %}
- **{{ name }}**
$${{ d.formulation }}$$
{% endfor %}
```

E FormulationBench Details

The FormulationBench dataset is a collection of 20 optimization problems, 109 MILP formulations, and 89 reformulation pairs (Table 4). The dataset can be downloaded via the `formulation-bench` Python package. The documentation[1] provides user guides, detailed problem and formulation descriptions, the dataset schema, and the package API reference. We provide a concise summary here. The dataset is comprised of three sources:

- **EquivaFormulation [68].** The five *EquivaFormulation* problems were sampled at random. These are simple optimization problems with a few scalar variables and constraints. Each one contains an original formulation and 10 transformations. These are enumerated in Table 1 of [68]. We make the following modifications.
  - The *Add Valid Inequalities* transformation is instance-specific. Because we are interested in formulation-level reformulation, we omit this transformation type.
  - We change the label on the *Replace by Base-10 Representation*. This formulation replaces each integer variable with a base-10 decimal expansion. It relies on using enough digit variables to support the size of the test instance data. Hence, this transformation is not valid under our formulation-level notion of reformulation.
- **EvoCut [66].** They define 7 optimization problems for which their method EvoCut proposes numerous cutting planes. We construct reformulation pairs by pairing the original MILP formulation with one augmented by the cut. A cutting plane valid for all instances is a valid reformulation.
- **Ferchtandiker et al. [16].** They define 8 real-world optimization problems, each admitting an efficient and inefficient formulation. We add each as a pair to *FormulationBench*.

Table 4: The 20 optimization problems in the FormulationBench dataset with their source. The *NP-hard* column indicates if the problem is NP-hard. We only use the 16 NP-hard problems in our experiments.

Problem	Name	Source	NP-hard
`p1`	EquivaFormulation Instance 47	EquivaFormulation [68]	✗
`p2`	EquivaFormulation Instance 74	EquivaFormulation [68]	✓
`p3`	EquivaFormulation Instance 92	EquivaFormulation [68]	✓
`p4`	EquivaFormulation Instance 183	EquivaFormulation [68]	✗
`p5`	EquivaFormulation Instance 217	EquivaFormulation [68]	✗
`p6`	Capacitated Warehouse Location (CWLP)	EvoCut [66]	✓
`p7`	Rectangular Tiling (IMO6)	EvoCut [66]	✗
`p8`	Job Shop Scheduling (JSSP)	EvoCut [66]	✓
`p9`	Multi-Commodity Network Design (MCND)	EvoCut [66]	✓
`p10`	Pickup and Delivery with Time Windows (PDPTW)	EvoCut [66]	✓
`p11`	Sub-Hour Unit Commitment (SHUC)	EvoCut [66]	✓
`p12`	Traveling Salesman Problem (TSP)	EvoCut [66]	✓
`p13`	Air Traffic Flow Management	Ferchtandiker et al. [16]	✓
`p14`	Blood Bank Netherlands	Ferchtandiker et al. [16]	✓
`p15`	Dutch Housing Problem	Ferchtandiker et al. [16]	✓
`p16`	Park and Bike Hub Location (Mobian)	Ferchtandiker et al. [16]	✓
`p17`	Open-Pit Mine Production Scheduling	Ferchtandiker et al. [16]	✓
`p18`	Timor-Leste Hospital Location	Ferchtandiker et al. [16]	✓
`p19`	UN Humanitarian Disaster Response Hub (UNHDR)	Ferchtandiker et al. [16]	✓
`p20`	World Food Program Food Distribution	Ferchtandiker et al. [16]	✓

Dataset Structure. Each problem contains (1) a Markdown problem description file, (2) a JSON information file, and (3) JSON files with instance data and the corresponding optimal solution. Each problem contains at least two formulations. Each formulation contains (1) a JSON information file, (2) a Python script to transform raw problem data into the formulation's parameter space, and (3) a ground-truth `MILPFormulation` Lean formalization. The JSON information file format is an extension of the format introduce by the NLP4LP dataset [1]. We add `assumptions` (see Section 6.1 for a discussion) and `definitions` fields. Lastly, each formulation is flagged as valid if it is faithful to the underlying optimization problem.

Reformulation Test Set. FormulationBench contains 89 pairs of formulations $(\mathcal{M}, \mathcal{M}')$ containing 63 positive examples where $\mathcal{M}'$ is a constructive reformulation of $\mathcal{M}$ and 26 negative examples. Our experiments use the 54 pairs (42 positive, 12 negative) belonging to the 16 NP-hard problems. Each reformulation pair additionally contains a JSON file specifying the parameter mapping Φ_{p} from $\mathcal{P}$ to $\mathcal{P}'$. For every valid reformulation, we provide a ground-truth `MILPReformulation` proof.

[1] https://formulation-bench.henryrobbins.com

F Invalid Reformulations

While preparing the FormulationBench dataset, we verified prior LLM-generated MILP reformulations: cutting planes proposed by EvoCut [66] and reformulation pairs proposed by Ferchtandiker et al. [16]. In the process, FLARE failed to produce reformulation certificates for 5 cutting planes and 4 formulations that were confirmed invalid upon manual inspection. In this section, we provide proofs of invalidity.

F.1 EvoCut Cutting Planes

Yazdani et al. [66] recently proposed the EvoCut framework to automatically generate *acceleration cuts* for MILP formulations using an LLM-based evolutionary search. Acceleration cuts are constraints added to a MILP formulation with the aim of reducing solve time. Importantly, such cuts are *not* guaranteed to be valid cutting planes or even optimality preserving. This pragmatic choice to consider acceleration cuts enables automation by reducing the computational burden required to verify candidate cuts. A cut is considered an acceleration cut if it does not change the optimal objective value on a small verification set of problem instances.

In the FormulationBench dataset, we constructed a reformulation from a cutting plane by adding the cut to the base formulation. A formulation constructed from a valid cutting plane trivially satisfies Definition 4.5. Using FLARE, we identified that 5 of the 43 acceleration cuts proposed by EvoCut are invalid cutting planes. These cuts span the TSP and a rectangle tiling problem. In the case of TSP, all three invalid cutting plane families are only invalid on TSP instances of size $n \leq 3$. While these counter-examples are not practically meaningful, they illustrate the importance of explicitly stating all assumptions of the problem data. In the case of rectangle tiling, the cuts meaningfully change the set of optimal solutions (see Figure 3). This motivates the importance of formally verifying reformulations, especially for non-standard optimization problems where an LLM is more likely to have an error in reasoning.

F.1.1 Traveling Salesman Problem (TSP)

For the TSP, EvoCut generates acceleration cuts for the Miller–Tucker–Zemlin (MTZ) formulation defined in Section 3. Between v1 and v2 of the arXiv preprint, there are 8 acceleration cuts proposed. FLARE identifies the following three as invalid cutting planes. Both cuts v1-EC3 and v2-EC2 eliminate triangles including the depot node and cut v2-EC1 eliminates all two-city subtours.

$$x_{j1} + x_{ji} + (u_j - u_i - 1) \leq (n-1)(2 - x_{1i} - x_{ij}) \qquad \forall i, j \in V \setminus \{1\},\ i \neq j \qquad \text{(v1-EC3)}$$
$$x_{ij} + x_{ji} \leq 1 \qquad \forall i, j \in V,\ i < j \qquad \text{(v2-EC1)}$$
$$x_{1i} + x_{i1} + x_{1j} + x_{j1} + x_{ij} + x_{ji} \leq 2 \qquad \forall i, j \in V \setminus \{1\},\ i < j \qquad \text{(v2-EC2)}$$

Proposition F.1. *The cut v1-EC3 is* not *a valid cutting plane for the MTZ TSP formulation.*

Proof. Consider the 3-node TSP instance depicted in Figure 2a with feasible tour $1 \to 2 \to 3 \to 1$. Letting $i = 2$ and $j = 3$, the cut v1-EC3 reduces to $1 \leq 0$, which does not hold. Hence, there exists a feasible integer point that is not satisfied by the cut. Therefore, the cut is invalid. □

Proposition F.2. *The cut v2-EC1 is* not *a valid cutting plane for the MTZ TSP formulation.*

Proof. Consider the 2-node TSP instance with the unique feasible tour $1 \to 2 \to 1$. The MTZ formulation permits $x_{12} = x_{21} = 1$, corresponding to traversing both arcs of this tour. Setting $i = 1$ and $j = 2$, the cut v2-EC1 reduces to $2 \leq 1$, which does not hold. Hence, there exists a feasible integer point that is not satisfied by the cut. Therefore, the cut is invalid. □

Proposition F.3. *The cut v2-EC2 is* not *a valid cutting plane for the MTZ TSP formulation.*

Proof. Consider the 3-node TSP instance depicted in Figure 2a with feasible tour $1 \to 2 \to 3 \to 1$. Setting $i = 2$ and $j = 3$, the cut v2-EC2 reduces to $3 \leq 2$, which does not hold. Hence, there exists a feasible integer point that is not satisfied by the cut. Therefore, the cut is invalid. □

F.1.2 Rectangular Tiling with One Hole per Row and Column (IMO6)

This problem is inspired by IMO 2025 Problem 6. Given an $N \times N$ grid of unit squares, one must place rectangular tiles (of various sizes) such that each row and column of the grid contain exactly one uncovered square (a *hole*). The objective is to minimize the number of tiles used. Yazdani et al. [66] introduce the following notation and formulation.

Notation.

- $R = \{1, \dots, N\}$ rows and $C = \{1, \dots, N\}$ columns
- $I = \{(a, b) \in C^2 \mid a \le b\}$ contiguous column-intervals
- $h_{ij} \in \{0, 1\}$ hole indicator
- $x_i^{ab} \in \{0, 1\}$ indicates if columns a through b on row i are covered by the same tile
- $s_i^{ab}, t_i^{ab} \in \{0, 1\}$ indicate if a tile spanning columns a to b begins on row i or ends on row i, respectively

Formulation.

$$
\begin{aligned}
\min \quad & \sum_{i \in R} \sum_{(a,b) \in I} s_i^{ab} \\
\text{s.t.} \quad & \sum_{j \in C} h_{ij} = 1 && \forall i \in R \\
& \sum_{i \in R} h_{ij} = 1 && \forall j \in C \\
& \sum_{\substack{(a,b) \in I \\ a \le j \le b}} x_i^{ab} + h_{ij} = 1 && \forall i \in R,\ \forall j \in C \\
& x_1^{ab} - s_1^{ab} = 0 && \forall (a, b) \in I \\
& x_i^{ab} - x_{i-1}^{ab} - s_i^{ab} + t_{i-1}^{ab} = 0 && \forall i = 2, \dots, N,\ \forall (a, b) \in I \\
& x_N^{ab} - t_N^{ab} = 0 && \forall (a, b) \in I \\
& h_{ij} \in \{0, 1\} && \forall i \in R,\ \forall j \in C \\
& x_i^{ab}, s_i^{ab}, t_i^{ab} \in \{0, 1\} && \forall i \in R,\ \forall (a, b) \in I
\end{aligned}
\tag{IMO6}
$$

Between `v1` and `v2` of the arXiv preprint, there are 8 acceleration cuts proposed. `FLARE` identifies the following two as invalid cutting planes. The cut `v1-EC1` enforces that a tile end (bottom-right corner) must be immediately to the left of any hole, and the cut `v1-EC2` enforces that a tile start (top-left corner) must be immediately to the right of any hole.

$$
h_{ij} \le \sum_{\substack{(a,b) \in I \\ b = j-1}} t_i^{ab} \qquad \forall i \in R,\ \forall j \in \{2, \dots, N\} \tag{v1-EC1}
$$

$$
h_{ij} \le \sum_{\substack{(a,b) \in I \\ a = j+1}} s_i^{ab} \qquad \forall i \in R,\ \forall j \in \{1, \dots, N-1\} \tag{v1-EC2}
$$

Proposition F.4. *The cut `v1-EC1` is* not *a valid cutting plane for the IMO6 formulation.*

Proof. Consider the $N = 3$ counterexample depicted in Figure 3. Let $i = j = 2$. The inequality reduces to $h_{22} \le t_2^{11}$. Substituting yields $1 \le 0$, which does not hold. Hence, there exists a feasible integer point that is not satisfied by the cut. Therefore, the cut is invalid. □

Proposition F.5. *The cut `v1-EC2` is* not *a valid cutting plane for the IMO6 formulation.*

Proof. Consider the $N = 3$ counterexample depicted in Figure 3. Let $i = 2$ and $j = 2$. The inequality reduces to $h_{22} \le s_2^{33}$. Substituting yields $1 \le 0$, which does not hold. Hence, there exists a feasible integer point that is not satisfied by the cut. Therefore, the cut is invalid. □

F.2 Ferchtandiker Formulation Pairs

Ferchtandiker et al. [16] introduces a dataset[1] containing an efficient and inefficient MILP formulation for 8 optimization problems. The dataset includes both LaTeX and GurobiPy code for each formulation. In the process of incorporating these reformulations in to the FormulationBench dataset, FLARE identified 4 invalid reformulations.

We prove the Air Traffic Management (ATM) and UN Humanitarian Disaster Response (UNHDR) reformulation pairs are not related by a constructive reformulation in either direction under the identity parameter mapping. We do not provide proofs for the World Food Program (WFP) since both formulations are polynomially-solvable linear programs, and our constructive reformulation definition is only meaningful on NP-hard formulations (Remark 4.6). The Open Pit Mining reformulation is intentionally a relaxation. It is expected to violate our definition and we omit any further discussion.

In FormulationBench, we modify the ATM, UNHDR, and WFP formulations to make them valid constructive reformulations, and we modify WFP to be NP-hard. See the documentation[2] for detailed descriptions of these modifications.

F.2.1 Air Traffic Management (ATM)

This problem consists of assigning each plane in an airline's fleet to a location (airport or airspace sector) over a sequence of time periods. The assignment must respect each location's capacity over time and the network's adjacency structure. Planes can only travel to locations that are a single time unit away. The objective is to maximize the total reward accrued for visiting locations. Ferchtandiker et al. [16] introduces the following formulations of this problem.

Notation.

- P: set of vehicles (planes).
- A: set of locations (airports and airspace sectors).
- T: set of time periods.
- $\text{cap}_{a,t}$: capacity of location $a \in A$ at time $t \in T$.
- $\tau_{a,a'}$: travel time from location a to location a'.
- $r_{a,t}$: reward for being at location $a \in A$ at time $t \in T$.

Formulations. Both formulations share the binary departure variable $x_{p,a,a',t} \in \{0,1\}$, which indicates whether plane p departs from a to a' at time t.[3] The Efficient formulation is purely event-based. The Inefficient formulation additionally tracks plane locations via $y_{p,a,t} \in \{0,1\}$, which indicates whether plane p is at location a at time t.

[1] The dataset is available at https://github.com/nathan-ferchtandiker/LLMs-For-Optimization-Reformulations

[2] https://formulation-bench.henryrobbins.com/en/latest/problems

[3] In the dataset, this variable is denoted $z_{p,a,a',t}$ in the inefficient formulation. We use $x_{p,a,a',t}$ for both formulations.

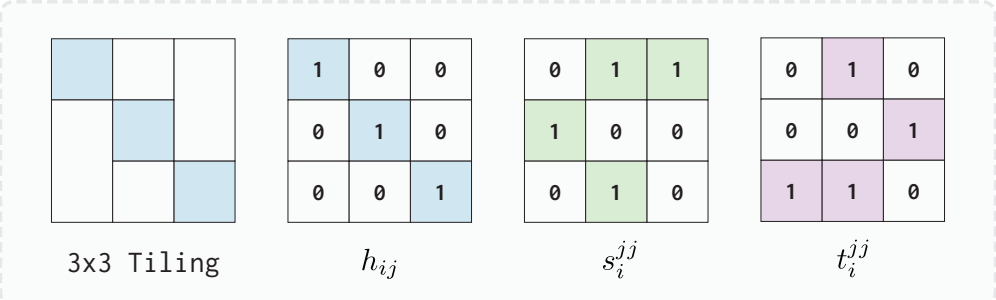

Figure 3: A feasible rectangular tiling for the $N = 3$ instance of IMO 2025 Problem 6. Decision variable values for the hole indicator h_{ij}, start flag s_i^{jj} and end flag t_i^{jj} are depicted in the grids. Note the figure does not specify s_i^{ab} and t_i^{ab} for $a < b$. However, since all tiles have unit width, these variables all have value 0.

$$
\begin{aligned}
\max \quad & \sum_{p\in P}\sum_{a\in A}\sum_{t\in T} r_{a,t}\, y_{p,a,t} && \\
\text{s.t.} \quad & \sum_{a\in A} y_{p,a,t} = 1 && \forall p\in P,\ t\in T \\
& \sum_{p\in P} y_{p,a,t} \le \mathrm{cap}_{a,t} && \forall a\in A,\ t\in T \\
& y_{p,a,t} = y_{p,a,t-1} + \\
& \quad \sum_{a'\in A}\sum_{t':t'+\tau_{a',a}=t} x_{p,a',a,t'} && \\
& \quad - \sum_{a'\in A} x_{p,a,a',t} && \forall p\in P,\ a\in A,\ t>0 \\
& y_{p,a,t}\in\{0,1\} && \forall p\in P,\ a\in A,\ t\in T \\
& x_{p,a,a',t}\in\{0,1\} && \forall p\in P,\ a,a'\in A,\ t\in T
\end{aligned}
$$

(Inefficient)

$$
\begin{aligned}
\max \quad & \sum_{p\in P}\sum_{a\in A}\sum_{t\in T} r_{a,t} \\
& \left(\sum_{a'\in A}\sum_{t':t'+\tau_{a',a}=t} x_{p,a',a,t'}\right) \\
\text{s.t.} \quad & \sum_{a\in A}\sum_{a'\in A}\sum_{t\in T} x_{p,a,a',t} \ge 1 && \forall p\in P \\
& \sum_{p\in P}\sum_{a'\in A} x_{p,a,a',t} \le \mathrm{cap}_{a,t} && \forall a\in A,\ t\in T \\
& x_{p,a,a',t}\in\{0,1\} && \forall p\in P,\ a,a'\in A,\ t\in T
\end{aligned}
$$

(Efficient)

The two formulations have misaligned objectives semantics. In the inefficient formulation, the reward $\sum_{p,a,t} r_{a,t}y_{p,a,t}$ accrues at every time period a plane is present at a location, so a plane that remains at a for k consecutive time periods contributes $\sum_t r_{a,t}$ over those k periods. In the efficient formulation, the reward only accrues on arrivals, so the same solution contributes only the single reward $r_{a,t}$ at the arrival time.

First, we establish a useful obstruction that asserts the existence of a bijection between the objective value sets of constructive reformulations.

Lemma F.6 (Objective-level bijection)**.** *Let $\Phi = (\Phi_{\mathrm{p}}, \Phi_{\mathrm{fwd}}, \Phi_{\mathrm{bwd}}, \Phi_{\mathrm{obj}})$ be a reformulation construction (Definition 4.5) from $\mathcal{M}$ to $\mathcal{M}'$, and let $p\in\mathcal{P}$ with $p'=\Phi_{\mathrm{p}}(p)$. Write $V(p)=\{f_0(x;p): x\in\mathcal{F}(p)\}$ and $V'(p')=\{f'_0(x';p'): x'\in\mathcal{F}'(p')\}$ for the attained objective-value sets. Then Φ_{obj} is a bijection from $V(p)$ onto $V'(p')$; in particular $|V(p)|=|V'(p')|$.*

Proof. We first show $\Phi_{\mathrm{obj}}(V(p))\subseteq V'(p')$. For $v=f_0(x;p)\in V(p)$ with $x\in\mathcal{F}(p)$, the point $\Phi_{\mathrm{fwd}}(x;p)$ lies in $\mathcal{F}'(p')$ by forward feasibility and has objective value $\Phi_{\mathrm{obj}}(v)$ by objective preservation. Hence $\Phi_{\mathrm{obj}}(v)\in V'(p')$. We now show the reverse inclusion $V'(p')\subseteq\Phi_{\mathrm{obj}}(V(p))$. For $v'=f'_0(x';p')\in V'(p')$ with $x'\in\mathcal{F}'(p')$, the point $x=\Phi_{\mathrm{bwd}}(x';p)$ lies in $\mathcal{F}(p)$ by backward feasibility and satisfies $v'=\Phi_{\mathrm{obj}}(f_0(x;p))$ by objective preservation. Hence $v'\in\Phi_{\mathrm{obj}}(V(p))$. Both inclusions yield $\Phi_{\mathrm{obj}}(V(p))=V'(p')$. Since Φ_{obj} is strictly monotonically increasing, it is injective, so this surjection is a bijection and $|V(p)|=|V'(p')|$. □

Proposition F.7. *The Inefficient and Efficient formulations of ATM are not related by a constructive reformulation (Definition 4.5) in either direction under the identity parameter mapping Φ_p.*

Proof. Consider a single-plane, single-location instance p. Let $P=\{1\}$, $A=\{a\}$, and $T=\{0,1\}$ with $\tau_{a,a}=1$, $\mathrm{cap}_{a,0}=\mathrm{cap}_{a,1}=1$, and $r_{a,0}=r_{a,1}=1$. Write $x_t:=x_{1,a,a,t}$ and $y_t:=y_{1,a,t}$. Let $V_{\text{eff}}(p)$ and $V_{\text{ineff}}(p)$ denote the objective values attained on p by the Efficient and Inefficient formulations respectively.

In the Efficient formulation, the constraints reduce to $x_0+x_1\ge 1$ and $x_t\le\mathrm{cap}_{a,t}=1$, so the feasible set is $\{(1,0),(0,1),(1,1)\}$. A departure at time t arrives at time $t+\tau_{a,a}=t+1$, so a departure at $t=1$ arrives at $t=2\notin T$ and contributes nothing; the objective equals $r_{a,1}x_0=x_0$. Hence $V_{\text{eff}}(p)=\{0,1\}$.

In the Inefficient formulation, $\sum_{a\in A} y_{1,a,t}=1$ with $|A|=1$ forces $y_0=y_1=1$ at every feasible point. The point $y_0=y_1=1$, $x_0=x_1=0$ satisfies the transition constraint, so the feasible set is nonempty. Its objective is $y_0r_{a,0}+y_1r_{a,1}=2$, giving $V_{\text{ineff}}(p)=\{2\}$.

Suppose that a constructive reformulation between the two formulations with Φ_{p} the identity exists. By Lemma F.6, $|V_{\text{eff}}(p)|=|V_{\text{ineff}}(p)|$ regardless of which formulation is the source. But $|V_{\text{eff}}(p)|=2\neq 1=|V_{\text{ineff}}(p)|$, a contradiction. □

F.2.2 UN Humanitarian Disaster Response Hub (UNHDR)

This problem consists of selecting a fixed number of response hubs to service a set of disaster regions. The selection must satisfy demand and response time constraints. The objective is to minimize transportation cost. Ferchtandiker et al. [16] introduces the following formulations of this problem.

Notation.

- H: set of candidate hubs.

- $H^f \subseteq H$: set of hubs that must be open (fixed hubs).
- C: set of disaster regions.
- a_c: number of people affected in region $c \in C$.
- C_{hc}: cost per person from hub h to region c.
- t_{hc}: travel time from hub h to region c.
- T: maximum allowed (weighted) travel time per region.
- n: maximum number of hubs that can be opened.
- M: sufficiently large constant (used in big-M constraints).

Formulations. The Inefficient formulation introduces a binary indicator variable y_h to indicate if hub $h \in H$ is opened and binary indicator z_{hc} to indicate if hub $h \in H$ is assigned to serve region $c \in C$. Lastly, x_{hc} is the fraction of region c's demand that is served by hub h.[1] The Efficient formulation drops the hub indicator z_{hc} variables.

$$
\begin{aligned}
\min \quad & \sum_{h\in H}\sum_{c\in C} a_c\, C_{hc}\, x_{hc} \\
\text{s.t.} \quad & x_{hc} \le M z_{hc} && \forall h \in H,\ c \in C \\
& \sum_{h\in H} z_{hc} = 1 && \forall c \in C \\
& \sum_{c\in C} z_{hc} \le |C| y_h && \forall h \in H \\
& \sum_{h\in H} x_{hc} = 1 && \forall c \in C \\
& \sum_{h\in H} y_h \le n \\
& y_h = 1 && \forall h \in H^f \\
& \sum_{h\in H} t_{hc} x_{hc} \le T && \forall c \in C \\
& x_{hc} \ge 0 && \forall h \in H,\ c \in C \\
& z_{hc} \in \{0,1\} && \forall h \in H,\ c \in C \\
& y_h \in \{0,1\} && \forall h \in H
\end{aligned}
\qquad \text{(Inefficient)}
$$

$$
\begin{aligned}
\min \quad & \sum_{h\in H}\sum_{c\in C} a_c C_{hc} x_{hc} \\
\text{s.t.} \quad & \sum_{c\in C} x_{hc} \le |C| y_h && \forall h \in H \\
& \sum_{h\in H} x_{hc} = 1 && \forall c \in C \\
& \sum_{h\in H} y_h \le n \\
& y_h = 1 && \forall h \in H^f \\
& \sum_{h\in H} t_{hc} x_{hc} \le T && \forall c \in C \\
& x_{hc} \ge 0 && \forall h \in H,\ c \in C \\
& y_h \in \{0,1\} && \forall h \in H
\end{aligned}
\qquad \text{(Efficient)}
$$

The two formulations differ in how the demand of a region may be met. In the Inefficient formulation, the assignment constraint $\sum_{h\in H} z_{hc} = 1$ together with the big-M constraint $x_{hc} \le M z_{hc}$ forces each region $c \in C$ to be served in full by a single hub. The Efficient formulation has no such restriction; a region can be supplied by a combination of multiple hubs simultaneously.

Proposition F.8. *The Inefficient and Efficient formulations of UNHDR are not related by a constructive reformulation (Definition 4.5) in either direction under the identity parameter mapping Φ_p.*

Proof. Consider the instance p with $H = \{1, 2\}$, $H^f = \emptyset$, $C = \{c\}$, $a_c = 1$, $n = 2$, costs $C_{1c} = 0$, $C_{2c} = 1$, travel times $t_{1c} = 2$, $t_{2c} = 0$, response time limit $T = 1$, and any $M \ge 1$. Let $V_{\text{eff}}(p)$ and $V_{\text{ineff}}(p)$ denote the objective values attained on p by the Efficient and Inefficient formulations respectively.

In the Efficient formulation, open both hubs ($y_1 = y_2 = 1$). The point $x_{1c} = \lambda$, $x_{2c} = 1 - \lambda$ is feasible for every $\lambda \in [0, \frac{1}{2}]$ (the response time constraint reads $2\lambda \le 1$) and attains objective value $1 - \lambda$. Hence $V_{\text{eff}}(p) \supseteq [\frac{1}{2}, 1]$ is uncountable.

In the Inefficient formulation, assigning region c to hub 1 ($z_{1c} = 1$) forces $x_{1c} = 1$ and violates the response time constraint $t_{1c} x_{1c} = 2 > 1 = T$. Hence the only feasible assignment is $z_{2c} = 1$, which forces $x_{2c} = 1$ and $y_2 = 1$, attaining objective value 1. Thus $V_{\text{ineff}}(p) = \{1\}$.

Suppose that a constructive reformulation between the two formulations with Φ_{p} the identity exists. By Lemma F.6, $|V_{\text{eff}}(p)| = |V_{\text{ineff}}(p)|$ regardless of which formulation is the source. But $V_{\text{eff}}(p)$ is uncountable while $|V_{\text{ineff}}(p)| = 1$, a contradiction. □

F.2.3 World Food Program (WFP)

This problem consists of designing a food distribution plan for the World Food Program (WFP) to deliver commodities from suppliers, through transshipment points, to beneficiary camps in crisis-affected regions. The plan must satisfy each camp's ration demand and meet per-person nutritional requirements across all nutrients. The objective is to minimize the total procurement and transportation cost. Ferchtandiker et al. [16] introduces the following formulations of this problem.

[1] In the dataset, this variable is denoted q_{hc}. We use x_{hc} for consistency with the efficient formulation.

Notation.

- K: set of commodities.
- L: set of nutrients.
- pc_k is the procurement cost per kg of commodity k.[1]
- $\mathrm{nutval}_{k\ell}$: nutrient-ℓ content (per kg) of commodity $k \in K$.
- nutreq_ℓ: per-person requirement for nutrient $\ell \in L$.
- dem_j: number of beneficiaries at camp j.
- $R_k \geq 0$: ration size (kg per person) of commodity $k \in K$.

Formulations. The Efficient formulation is node-based: N is the set of nodes, partitioned into suppliers N_S, transshipment points N_T, and beneficiary camps N_B. $E_{ij} \in \{0,1\}$ indicates whether an edge from i to j exists and we have a transportation cost $\mathrm{tc}_{ijk} \geq 0$ per kg of commodity k on edge (i,j). Edges run only from suppliers to transshipment points, between transshipment points, and from transshipment points to beneficiary camps; that is, $E_{ij} = 1$ only if

$$(i,j) \in (N_S \times N_T) \cup (N_T \times N_T) \cup (N_T \times N_B).$$

The decision variable $F_{ijk} \geq 0$ encodes the amount of commodity k shipped from i to j. The Inefficient formulation is path-based: P is the set of all simple paths from a supplier to a beneficiary camp. The indicator $e_{jp} \in \{0,1\}$ indicates whether path p ends at camp j and c_{pk} is the cost of shipping one kg of commodity k along path p. The decision variable $x_{pk} \geq 0$ is the amount of commodity k shipped along path p.

$$\begin{aligned}
\min \quad & \sum_{k\in K} pc_k \left(\sum_{p\in P} x_{pk} \right) + \sum_{p\in P} \sum_{k\in K} c_{pk}\, x_{pk} \\
\text{s.t.} \quad & \sum_{p\in P} e_{jp}\, x_{pk} \geq \mathrm{dem}_j\, R_k && \forall j \in N_B,\ k \in K \\
& \sum_{k\in K} \mathrm{nutval}_{k\ell}\, R_k \geq \mathrm{nutreq}_\ell && \forall \ell \in L \\
& x_{pk} \geq 0 && \forall p \in P,\ k \in K \\
& R_k \geq 0 && \forall k \in K
\end{aligned} \qquad \text{(Inefficient)}$$

$$\begin{aligned}
\min \quad & \sum_{k\in K} \mathrm{pc}_k \left(\sum_{j\in N_B} \mathrm{dem}_j R_k \right) + \sum_{i,j\in N} \sum_{k\in K} \mathrm{tc}_{ijk}\, F_{ijk} \\
\text{s.t.} \quad & \sum_{i\in N} E_{ij} F_{ijk} = \sum_{i\in N} E_{ji} F_{jik} && \forall j \in N_T,\ k \in K \\
& \sum_{i\in N} E_{ij} F_{ijk} \geq \mathrm{dem}_j R_k && \forall j \in N_B,\ k \in K \\
& \sum_{k\in K} \mathrm{nutval}_{k\ell} R_k \geq \mathrm{nutreq}_\ell && \forall \ell \in L \\
& F_{ijk} \geq 0 && \forall i,j \in N,\ k \in K \\
& R_k \geq 0 && \forall k \in K
\end{aligned} \qquad \text{(Efficient)}$$

Both WFP formulations are polynomially-solvable linear programs, but our constructive reformulation definition is only meaningful on NP-hard formulations (Remark 4.6). Despite this, FLARE still identified two inconsistencies between the pair of WFP formulations.

- **Misaligned objectives.** In the Efficient formulation, the procurement cost term $\sum_k \mathrm{pc}_k \sum_{j\in N_B} \mathrm{dem}_j R_k$ depends only on the ration size R_k, while the demand constraint $\sum_i E_{ij} F_{ijk} \geq \mathrm{dem}_j R_k$ permits shipping in excess of $\mathrm{dem}_j R_k$ without additional procurement penalty. In the Inefficient formulation, the procurement cost $\sum_k \mathrm{pc}_k \sum_p x_{pk}$ instead depends on the total amount shipped, so any slack in the demand constraint is charged.
- **Cycles.** In the Efficient formulation, we have flow variables F_{ijk}. A feasible point could have positive flow on a cycle of transshipment points. However, in the Inefficient formulation, P is the set of all *simple* paths from a supplier to a beneficiary camp. A feasible point necessarily has acyclic flow on each commodity.

Notably, these inconsistencies are only realized on sub-optimal feasible points. An optimal point will not ship excess demand nor send flow on a cycle. While the Efficient formulation is an Audet reformulation of the Inefficient formulation, our stricter constructive reformulation definition imposes conditions across the *entire* feasible region.

We modify WFP to resolve these inconsistencies for inclusion in FormulationBench and make the problem NP-hard by restricting to integral flows and adding transhipment throughput capacities.

[1] In the dataset, the variable q_k is used for procurement cost in the inefficient formulation. We use pc_k for both formulations.

G Experiment Details

Dataset. All experiments were run on `v0.5.0` of the `formulation-bench` Python package which pins the dataset to `v0.4.0`. See Appendix E and the package documentation for details.

FLARE agent harness. Experiments were conducted on a MacBook Pro (Apple M3 Pro, 12-core CPU, 18 GB unified memory) running macOS 15.7.1. We evaluate FLARE on the following agent harnesses:

- **Claude Code.** Version 2.1.197 with Claude Code Max subscription ($100/month)
- **Codex.** Version 0.147.0 with ChatGPT Pro subscription ($100/month)
- **OpenCode.** Version 1.18.15 (open-source)

We used Claude Code and ChatGPT subscriptions in order to avoid higher API costs. The FLARE experiments can be run within the weekly usage limits but must be batched in order to avoid the 5 hour Claude Code session limit.

FLARE compute. FLARE runs in a Docker container with a 30 minute time limit. The image is built on Ubuntu 24.04 with Python 3.12 and Node 20. It contains the three agent CLIs above, the Lean toolchain (`elan` 4.2.3 and Lean 4.28.0, with `mathlib` pinned to `v4.28.0`), and the `lean-lsp-mcp` 0.29.0 MCP server. To allow for increased parallelization, we run each FLARE in its own Modal[1] Sandbox. Each Sandbox is allocated a guaranteed floor of 2 CPU cores (it may burst higher) and 4 GiB of memory. Modal compute costs are excluded from the reported average costs. The compute costs of the FLARE experiments falls below the $30/month compute provided for free.

Table 5: Standard API rates (USD per million tokens) used to compute the reported average costs. *Input* is the uncached (cache-miss) rate and *Cached Input* the cache-hit rate; cache writes (billed at 1.25× input by Anthropic) are not modeled. GPT-4.1 publishes no cached rate, so its cached tokens are billed at the input rate. DeepSeek uses preview release rates prior to the 8-16-26 price increase.

Model	API Identifier	Input	Cached Input	Output
Opus 5	`claude-opus-5`	$5.000	$0.5000	$25.00
Sonnet 5	`claude-sonnet-5`	$2.000	$0.2000	$10.00
GPT-5.6 Sol	`gpt-5.6-sol`	$5.000	$0.5000	$30.00
GPT-5.6 Terra	`gpt-5.6-terra`	$2.000	$0.2000	$12.00
GPT-4.1	`gpt-4.1`	$2.000	—	$8.00
DeepSeek V4 Pro	`deepseek-v4-pro`	$0.435	$0.0036	$0.87
DeepSeek V4 Flash	`deepseek-v4-flash`	$0.140	$0.0028	$0.28

LLMs. Anthropic and OpenAI models are served through the Stanford AI API Gateway[2] at a discounted price. The reported average costs are computed using the standard API rates at the time of experimentation (Table 5). The gateway only supports structured output on OpenAI models (if reasoning is enabled). For Anthropic and DeepSeek models, we append the response schema to the prompt and configure retry logic if the response isn't parseable. We specify a max token budget of 8192 when reasoning is disabled and 16384 when reasoning is enabled. We use medium effort for Anthropic and OpenAI models and high effort for DeepSeek models. We found these settings produce comparable reasoning efforts across model families. DeepSeek models reason for longer, but *high* is the lowest effort available. When a model exhausts its token budget before emitting a verdict, we count the truncated response as a judgment of not a reformulation; this occurs only for DeepSeek V4 Pro, on 22 of its 486 ablation runs.

[1] https://modal.com
[2] https://uit.stanford.edu/service/ai-api-gateway

H Additional Results

Table 6: Evaluation of FLARE across agent harnesses and LLM models. Accuracy results on the FormulationBench dataset are provided in aggregate and segmented by source dataset. Metric cells report results from a single run and **Bold** indicates the best method on a metric. The Claude Code harness with Opus 5 is the only configuration with **100%** accuracy. Codex with GPT-5.6 Sol misses one pair. The open-source OpenCode harness with DeepSeek V4 Pro is an order of magnitude cheaper, but is substantially slower and less accurate. All 11 false negatives are attributable to ATP failure, 8 due to timeout limits (Appendix G).

Harness	Model	Effort	TP	FP	TN	FN	Precision	Recall	Accuracy	Avg. Time	Avg. Cost
Overall											
Claude Code	Opus 5	medium	42	0	12	0	**100.0%**	**100.0%**	**100.0%**	410.6s	$1.180
Codex	GPT-5.6 Sol	medium	41	0	12	1	**100.0%**	97.6%	98.1%	398.1s	$1.185
OpenCode	DeepSeek V4 Pro	high	31	0	12	11	**100.0%**	73.8%	79.6%	1366.9s	$0.127
EquivaFormulation [68] (p2, p3)											
Claude Code	Opus 5	medium	10	0	8	0	**100.0%**	**100.0%**	**100.0%**	229.8s	$0.628
Codex	GPT-5.6 Sol	medium	10	0	8	0	**100.0%**	**100.0%**	**100.0%**	253.6s	$0.722
OpenCode	DeepSeek V4 Pro	high	10	0	8	0	**100.0%**	**100.0%**	**100.0%**	370.7s	$0.031
EvoCut [66] (p6, p8–p12)											
Claude Code	Opus 5	medium	25	0	3	0	**100.0%**	**100.0%**	**100.0%**	413.4s	$1.116
Codex	GPT-5.6 Sol	medium	25	0	3	0	**100.0%**	**100.0%**	**100.0%**	478.1s	$1.436
OpenCode	DeepSeek V4 Pro	high	17	0	3	8	**100.0%**	68.0%	71.4%	1874.2s	$0.171
Ferchtandiker et al. [16] (p13–p20)											
Claude Code	Opus 5	medium	7	0	1	0	**100.0%**	**100.0%**	**100.0%**	807.5s	$2.645
Codex	GPT-5.6 Sol	medium	6	0	1	1	**100.0%**	85.7%	87.5%	443.6s	$1.348
OpenCode	DeepSeek V4 Pro	high	4	0	1	3	**100.0%**	57.1%	62.5%	1833.0s	$0.187

Table 7: Evaluation of FLARE-NL across LLM models and reasoning effort levels. Results show FLARE-NL evaluated on the FormulationBench dataset TSP problem. Metric cells report mean ± std. dev. across 3 runs; TP/FP/TN/FN reports totals. Every model achieves perfect accuracy with reasoning enabled; performance without reasoning varies widely across model families. To reduce costs, we only evaluate the leading frontier reasoning model from each provider in the ablation study (Table 3).

Model	Effort	TP	FP	TN	FN	Precision	Recall	Accuracy	Avg Time	Avg Cost
Reasoning Disabled										
Opus 5		15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	22.2s	$0.062
Sonnet 5		15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	38.9s	$0.042
GPT-5.6 Sol		15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	6.9s	$0.020
GPT-5.6 Terra		12	0	9	3	**100.0±0.0%**	80.0±20.0%	87.5±12.5%	5.2s	$0.008
GPT-4.1		0	0	9	15	—	0.0±0.0%	37.5±0.0%	6.5s	$0.008
DeepSeek V4 Pro		7	0	9	8	**100.0±0.0%**	46.7±11.5%	66.7±7.2%	6.4s	$0.001
DeepSeek V4 Flash		7	1	8	8	91.7±14.4%	46.7±23.1%	62.5±12.5%	5.0s	$0.0004
Reasoning Enabled										
Opus 5	medium	15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	16.5s	$0.049
Sonnet 5	medium	15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	23.4s	$0.029
GPT-5.6 Sol	medium	15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	12.8s	$0.027
GPT-5.6 Terra	medium	15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	10.6s	$0.012
DeepSeek V4 Pro	high	15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	65.8s	$0.005
DeepSeek V4 Flash	high	15	0	9	0	**100.0±0.0%**	**100.0±0.0%**	**100.0±0.0%**	46.3s	$0.002